

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN

ĐỒ ÁN 2

**Thiết kế và mô phỏng bộ lọc FIR thông thấp ứng dụng xử lý tín hiệu
ECG trên FPGA**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **LÊ PHONG VŨ**
MSSV: 22119157

TP. HỒ CHÍ MINH – 7/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN

ĐỒ ÁN 2

**Thiết kế và mô phỏng bộ lọc FIR thông thấp ứng dụng xử lý tín hiệu
ECG trên FPGA**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **LÊ PHONG VŨ**
MSSV: 22119157

Hướng dẫn: **PGS.TS. TRƯƠNG NGỌC SƠN**

TP. HỒ CHÍ MINH – 7/2026

BẢNG NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc đến quý thầy cô Khoa Điện – Điện tử đã tận tình giảng dạy, truyền đạt và trang bị cho em những kiến thức nền tảng vững chắc trong suốt quá trình học tập tại trường. Những kiến thức quý báu từ các môn học chuyên ngành không chỉ giúp em hiểu rõ hơn về lĩnh vực điện tử và kỹ thuật số, mà còn tạo tiền đề quan trọng để em có thể vận dụng một cách linh hoạt và hiệu quả vào thực tiễn cũng như trong toàn bộ quá trình thực hiện đề tài này.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc đến thầy Trương Ngọc Sơn - người đã trực tiếp hướng dẫn và đồng hành cùng em trong suốt quá trình thực hiện đồ án. Thầy đã tận tâm truyền đạt kiến thức chuyên môn, chia sẻ những kinh nghiệm thực tiễn quý giá, đồng thời đưa ra những nhận xét, góp ý và định hướng kịp thời trong từng giai đoạn của đề tài. Nhờ sự hỗ trợ và chỉ dẫn tận tình của thầy, em đã có thể xác định đúng hướng tiếp cận, giải quyết các vấn đề kỹ thuật phát sinh và hoàn thành đồ án một cách có hệ thống và hiệu quả.

Mặc dù em đã cố gắng hết sức trong quá trình nghiên cứu và thực hiện, song do giới hạn về mặt thời gian cũng như kinh nghiệm nghiên cứu còn chưa nhiều, báo cáo chắc chắn không thể tránh khỏi những thiếu sót và hạn chế nhất định. Em kính mong nhận được sự góp ý, nhận xét và chỉ dẫn thêm từ thầy cũng như quý thầy cô trong hội đồng, để từ đó em có thể rút ra những bài học kinh nghiệm quý báu, từng bước hoàn thiện kiến thức chuyên môn và nâng cao chất lượng nghiên cứu trong những đề tài tiếp theo.

Em xin chân thành cảm ơn.

MỤC LỤC

DANH MỤC HÌNH.....	6
DANH MỤC BẢNG	7
CÁC TỪ VIẾT TẮT.....	8
CHƯƠNG 1: GIỚI THIỆU	9
1.1 GIỚI THIỆU	9
1.2 MỤC TIÊU ĐỀ TÀI.....	9
1.3 GIỚI HẠN ĐỀ TÀI	10
1.4 PHƯƠNG PHÁP NGHIÊN CỨU	11
1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	12
1.6 BỐ CỤC QUYỀN BẢO CÁO	13
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	14
2.1 TỔNG QUAN VỀ TÍN HIỆU ĐIỆN TÂM ĐỒ (ECG)	14
2.1.1 BẢN CHẤT TÍN HIỆU.....	14
2.1.2 CÁC YẾU TỐ GÂY NHIỄU ECG	15
2.2 LÝ THUYẾT BỘ LỌC FIR	16
2.3 THIẾT KẾ BỘ LỌC FIR BẰNG PHƯƠNG PHÁP CỬA SỔ	17
2.4 LƯỢNG TỬ HÓA HỆ SỐ	18
2.5 LỢI THẾ CỦA FPGA TRONG XỬ LÝ TÍN HIỆU SỐ	18
2.6 CÁC PHƯƠNG PHÁP TỐI ƯU	19
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	20
3.1 THIẾT KẾ PHẦN CỨNG TRÊN VERILOG.....	20
3.1.1 CHỨC NĂNG PHẦN CỨNG.....	20
3.1.2 SƠ ĐỒ KHỐI PHẦN CỨNG.....	21
3.1.3 THIẾT KẾ TỪNG KHỐI.....	22

3.1.3.1 CHỌN THÔNG SỐ BỘ LỌC	24
3.1.3.2 TÍNH TOÁN HỆ SỐ BỘ LỌC	25
3.2 THIẾT KẾ PHẦN MỀM.....	29
3.2.1 CHỨC NĂNG HOẠT ĐỘNG CỦA PHẦN MỀM.....	29
3.2.2 LƯU ĐỒ GIẢI THUẬT	31
CHƯƠNG 4: KẾT QUẢ	33
4.1 HOẠT ĐỘNG CỦA HỆ THỐNG.....	33
4.1.1 MÔ HÌNH GIẢ LẬP TÍN HIỆU ECG.....	33
4.1.2 MÔ PHỎNG TÍN HIỆU ĐẦU RA	33
4.1.2.1 MÔ PHỎNG TÍN HIỆU ĐẦU RA VỚI VERILOG.....	33
4.1.2.2 MÔ PHỎNG TÍN HIỆU ĐẦU RA VỚI MATLAB	34
4.1.3 SO SÁNH KẾT QUẢ.....	35
4.2 ĐÁNH GIÁ PHÂN TÍCH HỆ THỐNG	36
4.2.1 PHÂN TÍCH KẾT QUẢ DỰA TRÊN SAI SỐ RMSE/MAE.....	36
4.2.2 PHÂN TÍCH CẤU HÌNH BỘ LỌC	37
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	40
5.1 KẾT LUẬN	40
5.2 HƯỚNG PHÁT TRIỂN	41
TÀI LIỆU THAM KHẢO	43
PHỤ LỤC	44
1. TẠO TÍN HIỆU GIẢ LẬP ECG	44
2. BỘ LỌC FIR THAM CHIẾU TRÊN MATLAB.....	45
3. BỘ LỌC FIR VERILOG	46
4. SO SÁNH HIỆU NĂNG	50

DANH MỤC HÌNH

Hình 2.1 Dạng sóng ECG chuẩn	14
Hình 3.1 Sơ đồ tổng quan của hệ thống	21
Hình 3.2 Sơ đồ thiết kế bộ lọc	22
Hình 3.3 Lưu đồ giải thuật tổng quan của hệ thống	31
Hình 3.4 Lưu đồ giải thuật khối nhân chập	32
Hình 3.5 Lưu đồ giải thuật khối giới hạn biên	32
Hình 4.1 Tín hiệu giả lập ECG	33
Hình 4.2 Tín hiệu giả lập sau khi thông qua bộ lọc (Verilog).....	34
Hình 4.3 Tín hiệu giả lập sau khi thông qua bộ lọc (Matlab).....	35
Hình 4.4 So sánh đánh giá kết quả	35
Hình 4.5 Biến đổi FFT trước và sau đối với bộ lọc 3 taps	37
Hình 4.6 Biến đổi FFT trước và sau đối với bộ lọc 5 taps	37
Hình 4.7 Biến đổi FFT trước và sau đối với bộ lọc 35 taps	38
Hình 4.8 Biến đổi FFT trước và sau đối với bộ lọc 39 taps	38

DANH MỤC BẢNG

Bảng 2.1 Các loại bộ lọc và chức năng	17
Bảng 2.2 Bảng so sánh các cửa sổ thông dụng.....	17
Bảng 3.1: Đáp ứng xung lý tưởng	26
Bảng 3.2: Cửa sổ Hamming	27
Bảng 3.3: Đáp ứng xung thực tế.....	27
Bảng 3.4: Đáp ứng xung thực tế đã chuẩn hóa.....	28
Bảng 3.5: Lượng tử hóa hệ số lọc.....	29
Bảng 4.1 Bảng so sánh sai số	36
Bảng 4.2 Bảng so sánh kết quả bộ lọc.....	39

CÁC TỪ VIẾT TẮT

ECG	Electrocardiogram
EMG	Electromyography
FIR	Finite Impulse Response
IIR	Infinite Impulse Response
FPGA	Field-Programmable Gate Array
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
FFT	Fast Fourier Transform
BW	Baseline Wander
MA	Motion Artifact
PLI	Power Line Interference
MAC	Multiply-Accumulate
ADC	Analog-to-Digital Converter
DAC	Digital-to-Analog Converter

CHƯƠNG 1: GIỚI THIỆU

1.1 GIỚI THIỆU

Trong y học hiện đại, tín hiệu điện tâm đồ (ECG) đóng vai trò then chốt trong chẩn đoán các bệnh lý tim mạch. Tuy nhiên, do biên độ tín hiệu ECG rất nhỏ — chỉ dao động trong khoảng 100 μV đến 5 mV — nên tín hiệu này cực kỳ nhạy cảm với nhiễu từ môi trường. Trong quá trình thu thập, các nguồn nhiễu phổ biến bao gồm nhiễu tần số lưới điện (50/60 Hz), nhiễu cơ (EMG) và dao động đường nền (Baseline Wander). Nếu không được xử lý triệt để, những thành phần nhiễu này sẽ làm sai lệch hình dạng sóng ECG và ảnh hưởng trực tiếp đến độ chính xác của kết quả chẩn đoán.

Để giải quyết vấn đề trên, tiền xử lý tín hiệu bằng các bộ lọc số là bước không thể bỏ qua. Trong số các lựa chọn hiện có, bộ lọc đáp ứng xung hữu hạn (FIR) được đánh giá là phù hợp nhất cho ứng dụng y sinh. Ưu điểm nổi bật của FIR là đặc tính pha tuyến tính, cho phép giữ nguyên hình dạng các sóng đặc trưng P, QRS và T sau khi lọc mà không gây méo dạng tín hiệu — điều mà các bộ lọc IIR khó đảm bảo được. Ngoài ra, FIR có cấu trúc không hồi tiếp nên luôn ổn định, tăng độ tin cậy cho toàn hệ thống.

Về phía hiện thực hóa phần cứng, công nghệ FPGA ngày càng được ưa chuộng nhờ kiến trúc xử lý song song. Không giống vi xử lý truyền thống vốn thực hiện lệnh tuần tự, FPGA cho phép triển khai đồng thời nhiều phép tính nhân-cộng, đáp ứng yêu cầu xử lý thời gian thực với độ trễ thấp và thông lượng cao.

Từ những yêu cầu thực tiễn và ưu thế kỹ thuật nêu trên, đề án "Thiết kế và mô phỏng bộ lọc FIR thông thấp ứng dụng xử lý tín hiệu ECG trên FPGA" được thực hiện nhằm làm chủ kỹ thuật thiết kế bộ lọc số và ứng dụng vào bài toán xử lý tín hiệu y sinh cụ thể.

1.2 MỤC TIÊU ĐỀ TÀI

Mục tiêu cốt lõi của đề án là xây dựng thành công một hệ thống lọc nhiễu cho tín hiệu ECG, bao gồm toàn bộ quy trình từ nghiên cứu lý thuyết, mô phỏng phần mềm cho đến hiện thực hóa trên phần cứng. Các mục tiêu cụ thể được triển khai theo từng giai đoạn như sau:

- Nghiên cứu nền tảng lý thuyết: Tìm hiểu đặc tính tín hiệu ECG, phân tích các thành phần nhiễu thường gặp và nắm vững lý thuyết thiết kế bộ lọc FIR, đặc biệt là phương pháp thiết kế sử dụng hàm cửa sổ (Window Method).
- Thiết kế và kiểm chứng trên MATLAB: Xây dựng mô hình bộ lọc chuẩn trên MATLAB để làm cơ sở tham chiếu. Tại giai đoạn này, tín hiệu ECG có nhiễu sẽ được giả lập để kiểm tra và đánh giá hiệu quả của thuật toán lọc trước khi chuyển sang hiện thực hóa phần cứng.
- Hiện thực hóa thông qua Verilog HDL: Chuyển đổi thuật toán từ MATLAB sang ngôn ngữ mô tả phần cứng Verilog HDL, hướng đến thiết kế một cấu trúc phần cứng tối ưu, đảm bảo tính chính xác bit-true so với mô hình phần mềm tham chiếu.
- Đánh giá và so sánh kết quả: So sánh kết quả đầu ra giữa mô phỏng MATLAB và mô phỏng qua Verilog nhằm khẳng định độ chính xác của thiết kế FPGA, đồng thời kiểm chứng khả năng loại bỏ nhiễu tần số cao của bộ lọc so với yêu cầu đặt ra ban đầu.

1.3 GIỚI HẠN ĐỀ TÀI

- Đề tài được thực hiện trong phạm vi mô phỏng phần mềm, chưa triển khai thực tế trên bo mạch FPGA vật lý. Toàn bộ quá trình thiết kế và kiểm chứng được thực hiện trên môi trường MATLAB và Xilinx ISE, chưa trải qua quy trình tổng hợp (Synthesis) hay phân tích định thời (Timing Analysis) trên một dòng chip cụ thể.
- Về dữ liệu đầu vào, đề tài không sử dụng tín hiệu ECG thực thu từ bệnh nhân mà chỉ dùng tín hiệu giả lập dạng sóng sin 1 Hz với biên độ vài trăm mV, không phản ánh đầy đủ đặc trưng lâm sàng thực tế như sóng P, phức hợp QRS hay sóng T. Dữ liệu đầu vào bị giới hạn ở định dạng Q1.7 signed 8-bit, có độ phân giải thấp hơn so với ADC y sinh thực tế thường dùng 12–16 bit. Về nhiễu, hệ thống chỉ xử lý được ba thành phần cơ bản gồm nhiễu cấu trúc 20 Hz, nhiễu Gaussian và nhiễu cao tần, chưa bao gồm nhiễu điện lưới 50/60 Hz, nhiễu cơ cơ EMG phức tạp hay nhiễu trôi đường nền dưới 0.5 Hz.

- Về chức năng hệ thống, đề tài chỉ thiết kế bộ lọc thông thấp với tần số cắt cố định 3 Hz và 29 taps, không hỗ trợ thay đổi thông số động hay phối hợp với các loại bộ lọc khác. Hệ số bộ lọc là cố định nên không có khả năng thích nghi theo sự biến đổi của nhiều môi trường. Ngoài ra, hệ thống chưa tích hợp các chức năng hậu xử lý lâm sàng như phát hiện nhịp tim hay cảnh báo rối loạn nhịp. Độ trễ nhóm cố định 14 mẫu (70 ms) cũng không được bù tự động, đòi hỏi phải căn chỉnh thủ công khi so sánh kết quả đầu ra với tín hiệu tham chiếu.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

- Phương pháp tổng hợp tài liệu lý thuyết: Tiến hành thu thập, đọc và hệ thống hóa các tài liệu chuyên ngành liên quan đến lĩnh vực nghiên cứu, bao gồm giáo trình, bài báo khoa học, tài liệu kỹ thuật và các công trình nghiên cứu trong và ngoài nước. Từ đó, nền tảng lý thuyết cần thiết được xây dựng và làm cơ sở định hướng cho toàn bộ quá trình thiết kế và triển khai đề tài.
- Phương pháp mô hình hóa và mô phỏng phần mềm: Đây là phương pháp trọng tâm của đề tài. MATLAB được sử dụng để xây dựng mô hình toán học của tín hiệu ECG giả lập, thiết kế bộ lọc FIR theo phương pháp cửa sổ Hamming, thực hiện lượng tử hóa hệ số và mô phỏng thuật toán lọc bit-true ở định dạng Fixed-point. Kết quả thu được từ MATLAB đóng vai trò là chuẩn tham chiếu để đối chiếu kết quả.
- Phương pháp hiện thực hóa và kiểm chứng phần cứng: Từ thuật toán đã được kiểm chứng trên MATLAB, quá trình chuyển đổi sang ngôn ngữ mô tả phần cứng Verilog HDL được thực hiện để hiện thực hóa bộ lọc FIR trên nền tảng FPGA. Quá trình mô phỏng chức năng được thực hiện trên môi trường Xilinx ISE, cho phép quan sát dạng sóng số, phân tích tài nguyên phần cứng và kiểm tra tính chính xác của từng khối logic trong thiết kế.
- Phương pháp đánh giá định lượng: Để đo lường khách quan hiệu quả của hệ thống, các chỉ số sai số định lượng phổ biến trong xử lý tín hiệu số là RMSE (Root Mean Square Error) và MAE (Mean Absolute Error) được áp dụng. Các chỉ số này được tính toán trên ba cặp so sánh: đầu ra Verilog với đầu ra MATLAB, đầu ra MATLAB với tín hiệu sạch gốc, và đầu ra Verilog với tín hiệu

sạch gốc. Bên cạnh đó, phân tích phổ tần số FFT cũng được sử dụng để đánh giá trực quan khả năng khử nhiễu của bộ lọc trong miền tần số.

- Phương pháp thực nghiệm đa kịch bản: Hệ thống được khảo sát qua nhiều kịch bản với các thông số bộ lọc khác nhau, cụ thể là thay đổi số lượng taps từ 3 đến 39. Việc so sánh kết quả giữa các kịch bản giúp xác định cấu hình tối ưu về mặt chất lượng lọc, tài nguyên phần cứng và độ trễ hệ thống, từ đó lựa chọn thông số thiết kế cuối cùng là 29 taps một cách có căn cứ thực nghiệm rõ ràng.

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

- Đề tài tập trung nghiên cứu các đối tượng chính sau:
- + Tín hiệu điện tâm đồ (ECG): Nghiên cứu đặc tính sinh lý, dải tần số hoạt động, các thành phần sóng đặc trưng và các loại nhiễu thường gặp trong quá trình thu thập tín hiệu ECG thực tế.
- + Bộ lọc số FIR thông thấp: Nghiên cứu cơ sở toán học, phương pháp thiết kế bằng cửa sổ Hamming, quá trình lượng tử hóa hệ số và các tiêu chí đánh giá chất lượng bộ lọc.
- + Công nghệ FPGA và ngôn ngữ Verilog HDL: Nghiên cứu kiến trúc phần cứng, quy trình hiện thực hóa thuật toán xử lý tín hiệu số trên nền tảng FPGA thông qua ngôn ngữ mô tả phần cứng Verilog.
- Phạm vi nghiên cứu:
- + Về tín hiệu: Sử dụng tín hiệu ECG giả lập dạng sóng sin 1 Hz được tạo ra bằng MATLAB, có chèn thêm nhiễu cao tần và nhiễu Gaussian để mô phỏng điều kiện thực tế, không sử dụng tín hiệu ECG thu trực tiếp từ bệnh nhân.
- + Về thiết kế bộ lọc: Thiết kế bộ lọc FIR thông thấp với tần số cắt 3 Hz, 29 taps, sử dụng phương pháp cửa sổ Hamming, định dạng Fixed-point Q1.15 cho hệ số và Q1.7 cho dữ liệu đầu vào.
- + Về nền tảng thực thi: Mô phỏng và kiểm chứng hệ thống trên phần mềm MATLAB và Xilinx ISE, chưa triển khai thực tế trên bo mạch FPGA vật lý.
- + Về đánh giá: Đánh giá hiệu năng hệ thống thông qua các chỉ số định lượng RMSE, MAE và phân tích phổ tần số FFT, giới hạn trong việc so sánh kết quả mô phỏng phần mềm với mô phỏng phần cứng.

1.6 BỐ CỤC QUYỀN BÁO CÁO

Chương 1: Giới thiệu

Chương 2: Cơ sở lý thuyết

Chương 3: Thiết kế hệ thống

Chương 4: Kết quả

Chương 5: Kết luận và hướng phát triển

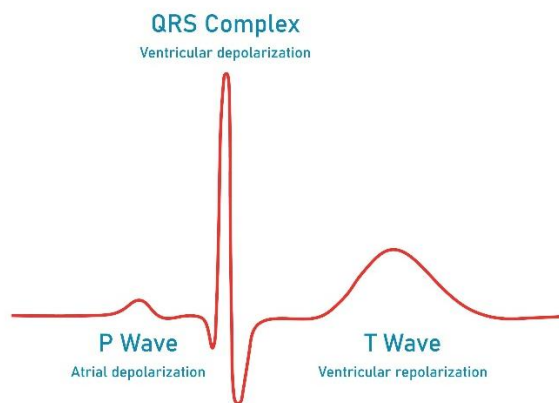
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Để xây dựng thành công bộ lọc số trên nền tảng FPGA, việc nắm vững đặc tính sinh lý của tín hiệu đầu vào và cơ sở toán học của thuật toán xử lý là yêu cầu tiên quyết. Chương này trình bày hệ thống các kiến thức nền tảng đã được nghiên cứu, bao gồm bản chất của tín hiệu điện tâm đồ (ECG), phân tích các nguồn nhiễu thực tế, lý thuyết thiết kế bộ lọc FIR, quy trình xử lý tín hiệu tổng quát và các vấn đề về lượng tử hóa khi hiện thực hóa trên phần cứng.

2.1 TỔNG QUAN VỀ TÍN HIỆU ĐIỆN TÂM ĐỒ (ECG)

2.1.1 BẢN CHẤT TÍN HIỆU

- Điện tâm đồ (Electrocardiogram – ECG) là đồ thị ghi lại sự biến thiên của các điện thế sinh học do tim phát ra trong quá trình hoạt động. Hệ thống dẫn truyền điện học của tim tạo ra các xung điện điều khiển quá trình co bóp cơ tim; các xung này có thể được đo không xâm lấn thông qua các điện cực đặt trên bề mặt cơ thể, từ đó phản ánh trạng thái sức khỏe của tim [1][2].
- Một chu kỳ ECG chuẩn bao gồm ba thành phần sóng cơ bản, tương ứng với ba giai đoạn hoạt động của tim:



Hình 2.1 Dạng sóng ECG chuẩn

- + Sóng P là thành phần đầu tiên trong chu kỳ, biểu thị quá trình khử cực tâm nhĩ. Sóng P thường có biên độ thấp, dưới 0,25 mV, và chứa các thành phần tần số thấp [1].

- + Phức hợp QRS là thành phần quan trọng nhất, biểu thị quá trình khử cực tâm thất. Đây là phần có biên độ lớn nhất, từ 1 đến 2 mV, với các thành phần tần số đặc trưng trong khoảng 10–40 Hz [1][3].
- + Sóng T biểu thị quá trình tái cực tâm thất trong giai đoạn hồi phục, thường có biên độ thấp hơn phức hợp QRS và kéo dài hơn về mặt thời gian [1].
- Phần thông tin hữu ích nhất cho chẩn đoán lâm sàng tập trung trong dải tần số từ 0,05 Hz đến 100 Hz [3]. Đây là tham số quan trọng để xác định tần số cắt cho các bộ lọc trong xử lý tín hiệu ECG.

2.1.2 CÁC YẾU TỐ GÂY NHIỄU ECG

Trong thực tế, tín hiệu ECG thu được thường bị ảnh hưởng bởi nhiều loại nhiễu khác nhau từ môi trường và cơ thể người, làm giảm chất lượng tín hiệu và gây khó khăn cho việc chẩn đoán. Ba loại nhiễu phổ biến và quan trọng nhất cần xử lý là nhiễu điện lưới, nhiễu cơ và trôi đường nền [2].

- Nhiễu điện lưới (Power Line Interference – PLI) là loại nhiễu xuất hiện do ảnh hưởng của điện từ trường phát ra từ lưới điện dân dụng xung quanh thiết bị đo. Loại nhiễu này có dạng sóng hình sin ổn định với tần số 50 Hz hoặc 60 Hz tùy theo từng quốc gia [1]. Dù thiết bị đo ECG được thiết kế để hạn chế loại nhiễu này, nhưng trên thực tế vẫn không thể loại bỏ hoàn toàn bằng phần cứng [2]. Nếu không được xử lý, PLI sẽ làm biến dạng sóng P và sóng T, khiến bác sĩ khó phân tích và đưa ra chẩn đoán chính xác [2].
- Nhiễu cơ (EMG Noise/Motion Artifact – MA) là loại nhiễu phát sinh từ sự co giãn của các cơ bắp trong quá trình đo, chẳng hạn khi bệnh nhân cử động hoặc run cơ. Đây là loại nhiễu khó xử lý nhất vì nó có tính chất không ổn định theo thời gian và trải rộng trên dải tần số từ 5 Hz đến 500 Hz, gần như chồng lấp hoàn toàn lên phổ tần số của tín hiệu ECG [2]. Biên độ của nhiễu này có thể lên đến 10% biên độ đỉnh-đỉnh của tín hiệu ECG, gây ra các biến dạng về hình dạng tín hiệu và ảnh hưởng trực tiếp đến kết quả phân tích [2].
- Trôi đường nền (Baseline Wander – BW) là hiện tượng đường cơ sở của tín hiệu ECG bị dịch chuyển lên xuống một cách chậm rãi theo thời gian, thay vì nằm ổn định trên một đường thẳng ngang. Nguyên nhân chính là do nhịp thở, cử động của bệnh nhân hoặc sự thay đổi điện trở tại vị trí tiếp xúc của điện cực [2].

Loại nhiễu này có tần số rất thấp, thường từ 0.15 Hz đến 0.3 Hz, và biên độ dao động khoảng 5% so với tín hiệu gốc [1][2]. Mặc dù trông có vẻ không đáng kể, nhưng BW có ảnh hưởng nghiêm trọng đến đoạn ST (đoạn nối từ cuối sóng S đến đầu sóng T) – một thành phần rất quan trọng trong việc chẩn đoán thiếu máu cơ tim và có thể làm mất hoặc che khuất các sóng biên độ nhỏ như sóng P và sóng T [2].

2.2 LÝ THUYẾT BỘ LỌC FIR

Bộ lọc FIR (Finite Impulse Response) là một lớp bộ lọc số tuyến tính bất biến thời gian (LTI) mà đáp ứng xung của nó có độ dài hữu hạn. Khác với bộ lọc IIR sử dụng phản hồi từ đầu ra, bộ lọc FIR chỉ phụ thuộc vào tín hiệu vào hiện tại và quá khứ, do đó luôn ổn định vô điều kiện [4]. Phương trình đầu ra của bộ lọc FIR bậc $N-1$ được mô tả bằng tích chập rời rạc:

$$y(n) = \sum_{k=0}^{N-1} h(k) \cdot x(n-k) \quad (2.0)$$

Trong đó:

$x(n)$: tín hiệu đầu vào

$y(n)$: tín hiệu đầu ra

$h(k)$: Là các hệ số của bộ lọc.

$x(n-k)$: Là các mẫu tín hiệu đầu vào tại thời điểm hiện tại và quá khứ

Để đạt pha tuyến tính, các hệ số phải thỏa điều kiện đối xứng qua tâm [6]:

$$h(n) = h(N-1-n) \quad (2.1)$$

- Tùy theo mục đích sử dụng, bộ lọc FIR được phân thành bốn loại cơ bản như bảng dưới đây [6][9]:

Loại bộ lọc	Chức năng
Thông thấp (Low-pass)	Giữ tần số thấp, chặn tần số cao
Thông cao (High-pass)	Giữ tần số cao, chặn tần số thấp
Thông dải (Band-pass)	Chỉ giữ một dải tần xác định

Chắn dải (Band-stop)	Loại bỏ một dải tần hẹp
----------------------	-------------------------

Bảng 2.1 Các loại bộ lọc và chức năng

- Đáp ứng xung lý tưởng $h_d(n)$ của bộ lọc thông thấp được xác định bằng hàm Sinc [6][9]:

$$h_d(n) = \frac{\sin(\omega_c(n - \alpha))}{\pi(n - \alpha)} \quad (2.2)$$

Trong đó $\alpha = (N-1)/2$ là độ trễ pha.

2.3 THIẾT KẾ BỘ LỌC FIR BẰNG PHƯƠNG PHÁP CỬA SỔ

Để hiện thực hóa bộ lọc FIR, đáp ứng xung lý tưởng vô hạn cần được cắt ngắn thành hữu hạn thông qua phương pháp cửa sổ(Windowing) [6]:

$$h(n) = h_d(n) \cdot w(n) \quad (2.3)$$

Loại cửa sổ	Độ rộng búp sóng chính	Độ suy giảm dải chặn (dB)	Nhận xét ứng dụng
Chữ nhật	$4\pi/N$	-13	Ít dùng cho ECG do nhiễu cao.
Tam giác	$8\pi/N$	-25	Đơn giản, dùng cho FPGA tài nguyên thấp.
Hamming	$8\pi/N$	-53	Tối ưu, cân bằng tốt giữa chất lượng và độ phức tạp.
Blackman	$12\pi/N$	-74	Khử nhiễu mạnh nhưng tốn tài nguyên tính toán.

Bảng 2.2 Bảng so sánh các cửa sổ thông dụng

- Cửa sổ Chữ nhật (Rectangular): Đơn giản nhất nhưng gây ra hiện tượng Gibbs với gợn sóng lớn, độ suy giảm dải chặn thấp ($\sim -13\text{dB}$) [6].
- Cửa sổ Tam giác (Bartlett): Giúp giảm gợn sóng nhờ biên độ giảm dần về hai đầu. Công thức đơn giản, phù hợp cho thiết kế tối ưu tài nguyên[6].
- Cửa sổ Hamming: Là loại cửa sổ tối ưu phổ biến nhất cho y sinh. Nó triệt tiêu tốt các búp sóng phụ, cho độ suy giảm dải chặn lớn ($\sim -53\text{dB}$) [6].
- Cửa sổ Blackman: Cung cấp khả năng khử nhiễu mạnh nhất ($\sim -74\text{dB}$) nhưng làm rộng dải chuyển tiếp[6].

Dựa trên bảng so sánh này, đồ án sẽ ưu tiên thử nghiệm cửa sổ Hamming để đạt chất lượng lọc tốt nhất.

2.4 LƯỢNG TỬ HÓA HỆ SỐ

- Lượng tử hóa là quá trình làm tròn các giá trị hệ số bộ lọc (thường rất nhỏ, ví dụ 0.0034) và tín hiệu đầu vào về các giá trị số nguyên tương ứng với độ dài bit cho trước (ví dụ: nhân với 2^8 hoặc 2^{16}) [11].

- Khi hiện thực hóa bộ lọc FIR trên FPGA, các hệ số dạng số thực phải được chuyển đổi sang định dạng số nguyên hữu hạn bit (Fixed-point) để phần cứng có thể xử lý. Quá trình lượng tử hóa hệ số, được thực hiện theo công thức [10]:

$$h_q(n) = \text{round}(h_{ideal}(n) \cdot 2^Q) \quad (2.4)$$

Trong đó Q là số bit phần thập phân. Đề tài sử dụng định dạng Q1.15 (1 bit số nguyên và 15 bit phần thập phân), tức $Q = 15$.

- Sai số phát sinh do lượng tử hóa được xác định bởi [10]:

$$e(n) = h_q(n) \cdot 2^{-Q} - h_{ideal}(n), \quad |e(n)| \leq \frac{1}{2^{15}} \quad (2.5)$$

Sai số này tuy nhỏ nhưng có thể làm thay đổi đáp ứng tần số thực tế của bộ lọc, biểu hiện qua việc giảm độ suy giảm dải chặn hoặc làm méo nhẹ tín hiệu trong dải thông. Do đó, việc chọn số bit đủ lớn là yếu tố quan trọng để đảm bảo chất lượng lọc sau khi triển khai trên phần cứng.

2.5 LỢI THẾ CỦA FPGA TRONG XỬ LÝ TÍN HIỆU SỐ

- So với các bộ xử lý tín hiệu số (DSP) truyền thống hoặc vi điều khiển, FPGA cung cấp những lợi thế vượt trội khi triển khai bộ lọc thông thấp:
- + Tính song song: FPGA cho phép thực hiện tất cả các phép nhân và phép cộng trong cấu trúc bộ lọc đồng thời trong một chu kỳ xung nhịp. Điều này đặc biệt quan trọng đối với các bộ lọc thông thấp bậc cao yêu cầu khối lượng tính toán lớn [9].
- + Tính linh hoạt: Khả năng tái cấu trúc giúp người dùng dễ dàng thay đổi các thông số bộ lọc như tần số cắt (f_c) hoặc độ gợn dải thông mà không cần thay đổi phần cứng vật lý [7].

- + Tốc độ xử lý: Nhờ kiến trúc đường ống (pipelining), FPGA có thể xử lý các tín hiệu có tốc độ lấy mẫu rất cao, đáp ứng các ứng dụng thời gian thực như xử lý âm thanh hoặc truyền thông số [5].

2.6 CÁC PHƯƠNG PHÁP TỐI ƯU

Trong quá trình triển khai trên FPGA, các kỹ thuật tối ưu hóa thường được áp dụng để cân bằng giữa hiệu suất và tài nguyên:

- + Sử dụng khối DSP chuyên dụng: Các dòng FPGA hiện đại (như Altera Cyclone hoặc Xilinx Virtex) tích hợp sẵn các khối DSP (DSP slices) chứa bộ nhân và bộ cộng tốc độ cao, giúp tiết kiệm tài nguyên logic tổng quát [4][9].
- + Tối ưu hóa công suất thấp: Sử dụng kiến trúc bộ nhân dựa trên bộ dồn kênh (Multiplexer-based shift/add multiplier) thay vì bộ nhân song song truyền thống giúp giảm đáng kể công suất tiêu thụ động và diện tích chip [8].
- + Lượng tử hóa hệ số: Vì FPGA hoạt động tốt nhất với số nguyên hoặc dấu phẩy tĩnh, các hệ số bộ lọc (thường là số thực từ thiết kế MATLAB) cần được lượng tử hóa thành các số nhị phân có độ dài bit cố định (ví dụ: 8-bit hoặc 16-bit) [4][6].

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

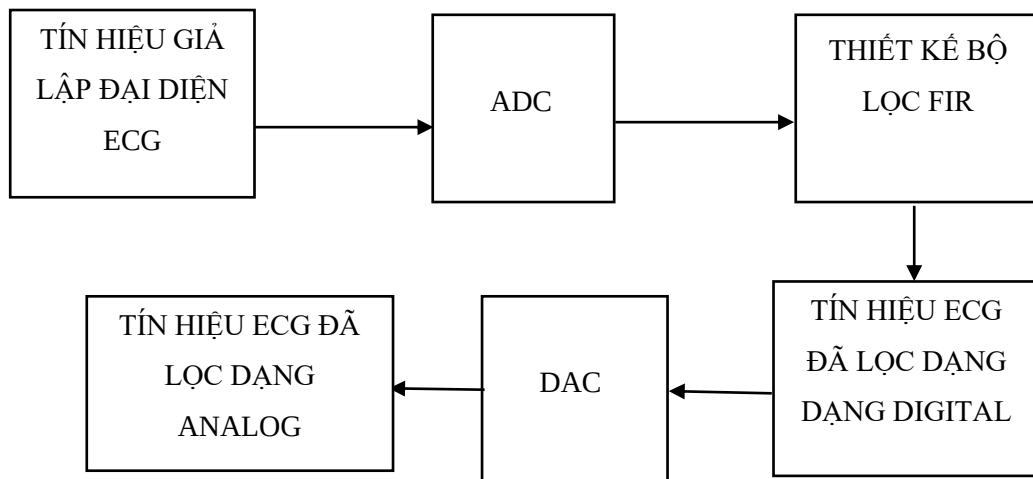
3.1 THIẾT KẾ PHẦN CỨNG TRÊN VERILOG

3.1.1 CHỨC NĂNG PHẦN CỨNG

- Ý tưởng thiết kế phần cứng được xây dựng nhằm hiện thực hóa thuật toán lọc nhiễu từ mô hình mô phỏng sang cấu trúc mạch logic có khả năng vận hành thời gian thực. Các chức năng chính bao gồm:

- + Chức năng số hóa và đồng bộ đầu vào: Tiếp nhận tín hiệu ECG (đã được giả lập) dưới dạng số nguyên có dấu 8-bit. Khối này đảm bảo dữ liệu đầu vào được đưa vào bộ lọc một cách tuần tự và đồng bộ hoàn toàn với xung clock, mô phỏng chính xác hoạt động của một bộ chuyển đổi ADC thực tế.
- + Chức năng lưu trữ và đáp ứng xung: Duy trì 29 giá trị hệ số bộ lọc đã được tối ưu hóa bằng phương pháp hàm cửa sổ từ MATLAB. Các hệ số này được lưu trữ cố định trong các thanh ghi với độ phân giải 16-bit (Q1.15), đóng vai trò quyết định đặc tính thông thấp để loại bỏ các thành phần nhiễu tần số cao trên 3 Hz.
- + Chức năng xử lý tín hiệu song song (MAC Unit): Đây là chức năng cốt lõi, thực hiện đồng thời các phép nhân mẫu tín hiệu với hệ số và cộng tích lũy kết quả (Multiply-Accumulate). Khối chức năng này tận dụng ưu thế của FPGA để tính toán toàn bộ bậc của bộ lọc chỉ trong một chu kỳ xung nhịp, đảm bảo tính thời gian thực cho việc theo dõi tín hiệu y tế.
- + Chức năng quản lý dải động và chuẩn hóa: Giám sát quá trình tính toán thông qua bộ tích lũy 32-bit để tránh hiện tượng tràn số. Sau khi tính toán, hệ thống thực hiện chức năng cắt tía để trích xuất 16-bit dữ liệu có giá trị nhất, giúp tái tạo dạng sóng ECG sạch nhiễu với biên độ chuẩn xác, phục vụ cho việc so sánh với mô hình tham chiếu.
- + Chức năng kiểm soát trạng thái hệ thống (Reset): Cung cấp khả năng tái lập trạng thái phần cứng, đưa toàn bộ các thanh ghi dịch và bộ đếm về giá trị không (0), đảm bảo hệ thống bắt đầu quá trình lọc từ trạng thái sạch, không bị ảnh hưởng bởi dữ liệu rác từ các phiên làm việc trước đó.

3.1.2 SƠ ĐỒ KHỐI PHẦN CỨNG



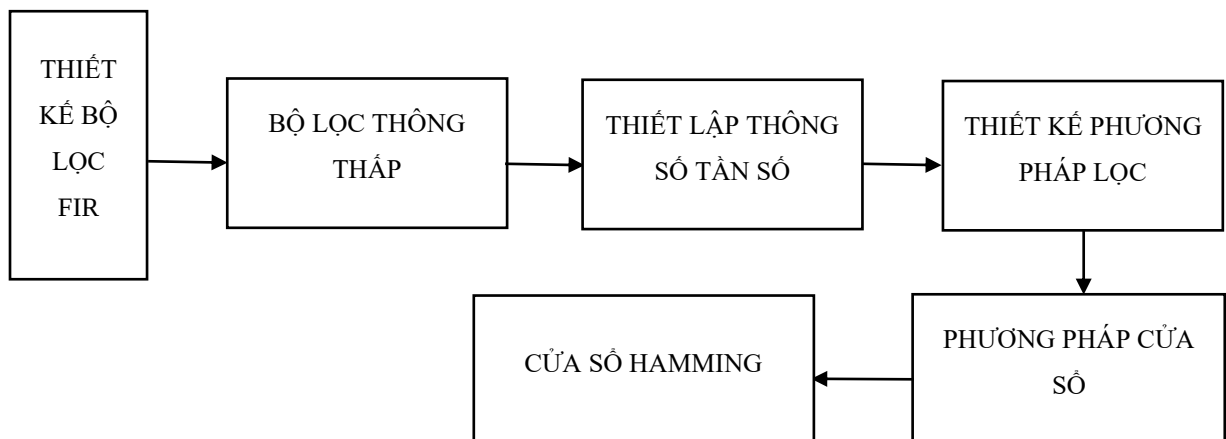
Hình 3.1 Sơ đồ tổng quan của hệ thống

Chức năng của từng khối:

- Tín hiệu giả lập đại diện ECG:
 - + Nguồn gốc: Tín hiệu được tạo ra bằng MATLAB, dạng sóng sin 1 Hz để minh họa đặc trưng nhịp tim [3][9].
 - + Biên độ: khoảng vài trăm mV (giả lập), không phải biên độ cực nhỏ như ECG thực. Đặc trưng: chỉ có một thành phần tần số cơ bản (1 Hz), không có các sóng P–QRS–T [3][9].
 - + Nhiều: được chèn thêm thành phần nhiễu tần số cao để kiểm chứng khả năng lọc [3][9].
 - + Ý nghĩa: tín hiệu này đóng vai trò đại diện cho ECG, dùng để kiểm tra nguyên lý lọc FIR [3][9].
- Khối ADC (Analog-to-Digital Converter):
 - + Chức năng: Chuyển đổi tín hiệu analog (ECG thực hoặc tín hiệu giả lập dạng sóng sin) thành tín hiệu số để FPGA xử lý [9].
 - + Trong thực tế: ADC lấy mẫu theo tần số F_s , lượng tử hóa biên độ thành số nguyên theo độ phân giải (ví dụ 12-bit, 16-bit) [9].
 - + Trong mô phỏng Verilog: bạn xuất input hex từ MATLAB → đây chính là dữ liệu số đã được “ADC hóa” [9].

- + Ý nghĩa: đảm bảo tín hiệu analog được biểu diễn dưới dạng số nhị phân/hex để đưa vào bộ lọc FIR [9].
- Khối DAC (Digital-to-Analog Converter):
- + Chức năng: Chuyển đổi tín hiệu số sau khi đã được bộ lọc FIR xử lý (tín hiệu sạch nhiễu) trở lại thành tín hiệu điện analog để hiển thị trên các thiết bị ngoại vi như máy hiện sóng (Oscilloscope) hoặc thiết bị y tế [5][9].
- + Trong thực tế: DAC nhận các mẫu tín hiệu số từ FPGA theo từng chu kỳ xung nhịp, thực hiện nội suy và chuyển đổi chúng thành các mức điện áp tương ứng dựa trên độ phân giải của chip DAC [5].
- + Trong mô phỏng Verilog: Dữ liệu đầu ra của bộ lọc thường được ghi lại dưới dạng tệp tin (.txt hoặc .hex) để đưa ngược lại MATLAB nhằm vẽ đồ thị và kiểm tra đáp ứng tần số → Đây là quá trình "tái tạo tín hiệu" trong môi trường mô phỏng [4][7].
- + Ý nghĩa: Là mắt xích cuối cùng giúp đưa kết quả xử lý số từ môi trường thuật toán ra thế giới thực, cho phép bác sĩ hoặc người dùng quan sát được dạng sóng ECG trực quan và liên tục [5].

3.1.3 THIẾT KẾ TỪNG KHỐI



Hình 3.2 Sơ đồ thiết kế bộ lọc

- Thiết kế bộ lọc FIR:

- + Chức năng: xác định mục tiêu thiết kế một bộ lọc số có đáp ứng xung hữu hạn (FIR) [9].
- + Ưu điểm: pha tuyến tính, giúp tín hiệu sau lọc không bị mất đi hình dạng ban đầu [9].
- + Ý nghĩa: đảm bảo tín hiệu giả lập sau lọc vẫn giữ nguyên hình dạng sóng sin cơ bản [9].
- Bộ lọc thông thấp:
 - + Chức năng: loại bỏ các thành phần tần số cao không mong muốn trong tín hiệu giả lập [9].
 - + Khử nhiễu: giảm nhiễu cao tần được chèn vào tín hiệu [9].
 - + Làm trơn tín hiệu: giữ lại thành phần 1 Hz, loại bỏ biến thiên nhanh [9].
 - + Ý nghĩa: minh họa nguyên lý lọc nhiễu, dù tín hiệu không phải ECG thực [9].
- Thiết lập thông số tần số:
 - + Tần số cắt (F_c): chọn khoảng 3 Hz để giữ lại thành phần cơ bản 1 Hz, loại bỏ nhiễu cao tần [9].
 - + Tần số lấy mẫu (F_s): 200 Hz, đảm bảo tín hiệu được số hóa đầy đủ [9].
 - + Các thông số khác phụ thuộc vào F_s và yêu cầu lọc [9].
- Phương pháp thiết kế bộ lọc: Sau khi cân nhắc, xem xét thì phương pháp cửa sổ là phù hợp nhất với ưu điểm là đơn giản, không tiêu tốn quá nhiều tài nguyên phần cứng của FPGA .

[Phương pháp cửa sổ]

Bộ đệm dữ liệu: Khi tính $y(n)$, cần cả giá trị hiện tại $x(n)$ lẫn các mẫu trước đó $x(n-1), x(n-2), \dots$. Bộ đệm (buffer) chính là thứ giữ lại các mẫu cũ, đưa vào khối nhân [6][9].

Khối nhân hệ số: Đây là nơi mỗi mẫu dữ liệu được nhân với một hệ số lọc $h[k]$ [6][9].

Đầu tiên, ta tính đáp ứng lý tưởng của bộ lọc (dài vô tận) [6][9].

Sau đó, cắt gọn bằng một hàm cửa sổ $w[k]$ (Hamming, Hanning, Blackman...) → để giảm rò rỉ phổ [6][9].

Kết quả:
$$h[k] = h_{lt}[k] \times w[k] \quad (3.0)$$

Khởi tổng:

Sau khi nhân từng mẫu với hệ số tương ứng, cộng tất cả lại [6][9] :

$$y(n) = \sum_{k=0}^{N-1} h(k) \times x(n - k) \quad (3.1)$$

Đây chính là tín hiệu đã lọc tại thời điểm n .

+ Cửa sổ Hamming:

Chức năng:

Trong thiết kế bộ lọc FIR bằng phương pháp cửa sổ, ta cần một hàm cửa sổ để cắt ngắn đáp ứng lý tưởng (vốn dài vô hạn). Hàm Hamming là một trong những lựa chọn phổ biến vì nó có khả năng giảm rò rỉ phổ (spectral leakage) tốt hơn so với cửa sổ hình chữ nhật, đồng thời giữ được độ rộng băng thông chuyển tiếp hợp lý. Khi áp dụng, đáp ứng lý tưởng $h_d(n)$ sẽ được nhân với hàm cửa sổ Hamming $w(n)$ để tạo ra hệ số lọc thực tế $h(n)$ [6][9].

Công thức [6] :

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), 0 \leq n \leq N-1 \quad (3.2)$$

Trong đó:

N : số tap của bộ lọc (ví dụ đang triển khai 29 tap).

n : chỉ số mẫu.

$w(n)$: giá trị cửa sổ Hamming tại vị trí n .

Ý nghĩa:

- Giảm biên độ của các thùy phụ (side lobes) trong miền tần số, giúp hạn chế nhiễu ngoài dải lọc [6][9].
- Giữ cho thùy chính (main lobe) đủ hẹp để phân biệt rõ vùng tần số cần giữ lại và vùng cần loại bỏ [6][9].
- Đảm bảo bộ lọc FIR sau khi thiết kế có đáp ứng mượt, giảm các gợn sóng [6][9].

3.1.3.1 CHỌN THÔNG SỐ BỘ LỌC

- Tần số cắt (Cut-off Frequency):

Đối với tín hiệu giả lập đại diện ECG trong đồ án, thành phần hữu ích chính nằm ở tần số thấp (khoảng 1 Hz). Vì vậy, bộ lọc được thiết kế theo dạng thông thấp với tần số cắt khoảng 3 Hz. Giá trị này đảm bảo giữ lại thành phần cơ bản của tín

hiệu, đồng thời loại bỏ nhiễu cao tần được chèn vào. Việc lựa chọn tần số cắt thấp cũng giúp minh họa rõ ràng khả năng lọc của FIR trong môi trường mô phỏng [6][9].

- Số tap (Filter Length):

Số tap quyết định độ sắc nét của đáp ứng tần số và độ rộng thùy chính trong miền tần số. Trong đồ án, số tap được chọn là 29. Đây là giá trị cân bằng giữa chất lượng lọc và độ phức tạp phần cứng. Với 29 tap, bộ lọc có độ trễ nhóm là 14 mẫu, một giá trị chấp nhận được trong mô phỏng và dễ dàng bù trễ khi so sánh kết quả MATLAB – Verilog. Nếu số tap quá nhỏ, đáp ứng lọc sẽ kém chính xác; ngược lại, nếu quá lớn, tài nguyên phần cứng FPGA sẽ bị tiêu tốn nhiều [6][9].

- Cửa sổ (Window Function):

Trong các loại cửa sổ phổ biến như Rectangular, Hanning, Blackman, cửa sổ Hamming được lựa chọn. Ưu điểm của Hamming là giảm biên độ thùy phụ, hạn chế rò rỉ phổ, đồng thời vẫn giữ được độ rộng thùy chính hợp lý. Điều này giúp bộ lọc FIR có đáp ứng mượt, ít dao động Gibbs, và dễ hiện thực hóa trên FPGA. Việc chọn cửa sổ Hamming cũng phù hợp với mục tiêu minh họa nguyên lý lọc trong môi trường giả lập [6][9].

3.1.3.2 TÍNH TOÁN HỆ SỐ BỘ LỌC

Đáp ứng xung lý tưởng (low-pass):

- $f_c = 3 \text{ Hz}$, $F_s = 200 \text{ Hz}$

→ $f_c/F_s = 0.015$

- $N=29$

→ $n_0 = (N-1)/2 = 14$

- Đáp ứng xung lý tưởng:

$$h_{\text{ideal}}(k) = \frac{\sin\left(\frac{2\pi \cdot f_c}{F_s} \cdot (k - n_0)\right)}{\pi \cdot (k - n_0)} \quad (3.3)$$

$$h_d(n_0) = 2 \frac{f_c}{F_s} \quad (3.4)$$

Hệ số k	Đáp ứng xung lý tưởng
0 và 28	0.022022
1 và 27	0.023038
2 và 26	0.024001
3 và 25	0.024908
4 và 24	0.025752
5 và 23	0.026530
6 và 22	0.027237
7 và 21	0.027871
8 và 20	0.028426
9 và 19	0.028902
10 và 18	0.029294
11 và 17	0.029602
12 và 16	0.029823
13 và 15	0.029956
14	0.030000

Bảng 3.1: Đáp ứng xung lý tưởng

Cửa sổ Hamming:

Theo như công thức (3.2): $w_{\text{ham}}(k) = 0,54 - 0,46 \cdot \cos\left(\frac{2\pi k}{N-1}\right)$

Hệ số k	Cửa sổ Hamming
0 và 28	0.080000
1 và 27	0.091533
2 và 26	0.125554
3 và 25	0.180358
4 và 24	0.253195
5 và 23	0.340413
6 và 22	0.437640
7 và 21	0.540000
8 và 20	0.642360
9 và 19	0.739587

10 và 18	0.826805
11 và 17	0.899642
12 và 16	0.954446
13 và 15	0.988467
14	1.000000

Bảng 3.2: Cửa sổ Hamming

Đáp ứng xung thực tế:

$$h_k = h_{\text{ideal}}(k) \times w_{\text{ham}}(k) \quad (3.5)$$

Hệ số k	Đáp ứng xung lý tưởng	Cửa sổ Hamming	Đáp ứng xung thực tế
0 và 28	0.022022	0.080000	0.001762
1 và 27	0.023038	0.091533	0.002108
2 và 26	0.024001	0.125554	0.003013
3 và 25	0.024908	0.180358	0.004493
4 và 24	0.025752	0.253195	0.006520
5 và 23	0.026530	0.340413	0.009033
6 và 22	0.027237	0.437640	0.011916
7 và 21	0.027871	0.540000	0.015050
8 và 20	0.028426	0.642360	0.018260
9 và 19	0.028902	0.739587	0.021378
10 và 18	0.029294	0.826805	0.024215
11 và 17	0.029602	0.899642	0.026639
12 và 16	0.029823	0.954446	0.028458
13 và 15	0.029956	0.988467	0.029610
14	0.030000	1.000000	0.030000

Bảng 3.3: Đáp ứng xung thực tế

Chuẩn hóa độ lợi DC

Mục tiêu: Đảm bảo tổng hệ số gần 1 để độ lợi DC bằng 1, tín hiệu mức một không bị khuếch đại/suy giảm [6][9].

$$G_{DC} = \sum_{k=0}^{N-1} h(k) \quad (3.6)$$

$$h_{\text{norm}}(k) = \frac{h(k)}{G_{DC}} \quad (3.7)$$

$$G_{DC} \approx 0.435$$

Hệ quả [6][9]:

$$H(e^{j0}) = \sum_{n=0}^{N-1} h_{norm}(n) \approx 1 \quad (3.8)$$

Hệ số k	Đáp ứng xung chuẩn hóa
0 và 28	0.004051
1 và 27	0.004846
2 và 26	0.006927
3 và 25	0.010329
4 và 24	0.014990
5 và 23	0.020765
6 và 22	0.027399
7 và 21	0.034600
8 và 20	0.041979
9 và 19	0.049146
10 và 18	0.055681
11 và 17	0.061246
12 và 16	0.065428
13 và 15	0.068077
14	0.068970

Bảng 3.4: Đáp ứng xung thực tế đã chuẩn hóa

Lượng tử hóa sang Q1.15 (fixed-point):

$$h_{q15}(k) = \text{round}(h_{norm}(k) \times 2^{15}) \quad (3.9)$$

Hệ số k	Giá trị thập phân	Lượng tử hóa	Giá trị thập lục phân
0 và 28	0.004051	133	0x0085
1 và 27	0.004846	159	0x009F
2 và 26	0.006927	227	0x00E3
3 và 25	0.010329	338	0x0152
4 và 24	0.014990	491	0x01EB
5 và 23	0.020765	680	0x02A8
6 và 22	0.027399	898	0x0382

7 và 21	0.034600	1134	0x046E
8 và 20	0.041979	1376	0x0560
9 và 19	0.049146	1610	0x064A
10 và 18	0.055681	1825	0x0721
11 và 17	0.061246	2006	0x07D6
12 và 16	0.065428	2145	0x0861
13 và 15	0.068077	2231	0x08B7
14	0.068970	2260	0x08D4

Bảng 3.5: Lượng tử hóa hệ số lọc

Phép tích lũy MAC và căn chỉnh bit

Nhân – cộng (MAC):

$$p_k(n) = x(n-k) \times h(n) \quad (Q2.22)$$

Đầu vào là Q1.7 $\rightarrow$ nhân với Q1.15 $\rightarrow$ được Q2.22

Muốn ra Q1.15 $\rightarrow$ cần dịch phải 7 bit (bỏ 7 bit thập phân thừa).

$$y_{fix}(n) = p_k(n) \gg 7$$

Chống tràn:

$$y_{fix}(n) = \begin{cases} 32767, & y_{fix}(n) > 32767 \\ -32767, & y_{fix}(n) < -32767 \\ y_{fix}(n), & \text{ngược lại} \end{cases} \quad (3.10)$$

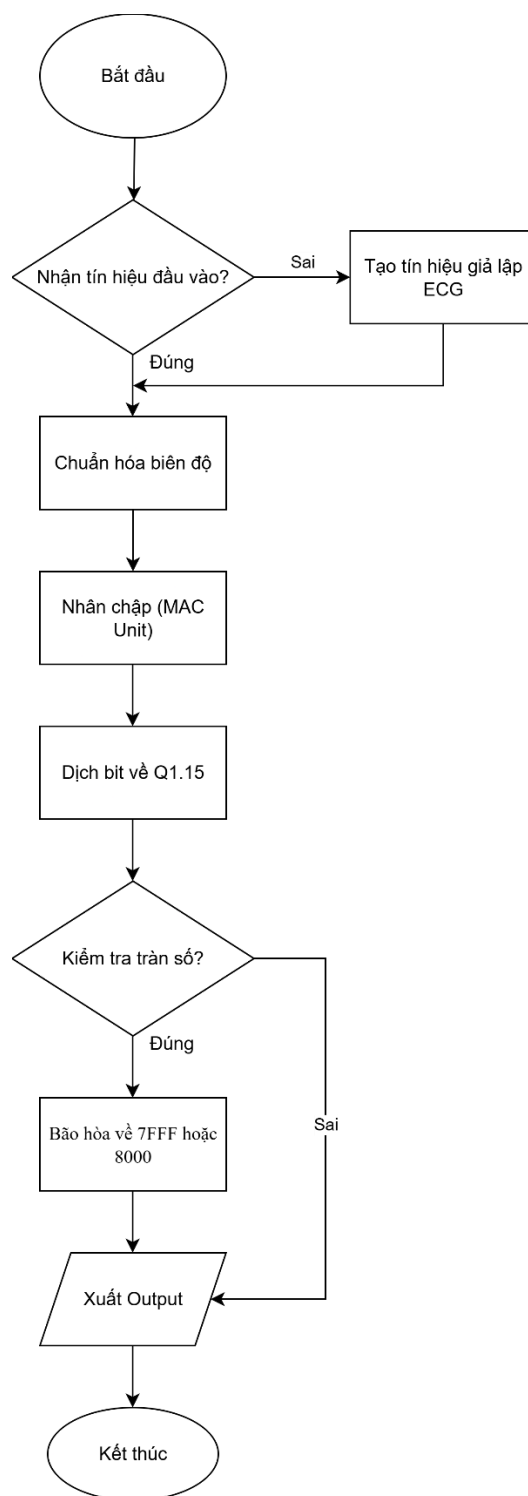
3.2 THIẾT KẾ PHẦN MỀM

3.2.1 CHỨC NĂNG HOẠT ĐỘNG CỦA PHẦN MỀM

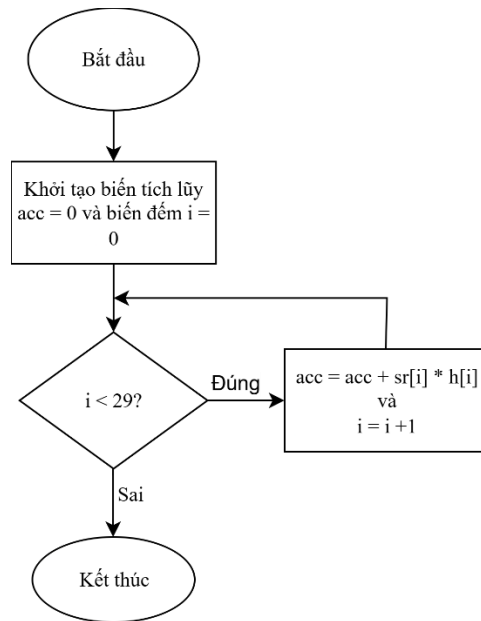
- Vai trò và chức năng của MATLAB: MATLAB đóng vai trò là môi trường thiết kế, giả lập và phân tích hệ thống ở mức độ thuật toán. Các chức năng chính bao gồm:
 - + Tạo tín hiệu đầu vào: Phần mềm tạo ra tín hiệu ECG giả lập (sóng sin 1Hz) và chèn thêm các thành phần nhiễu (nhiều cấu trúc 20Hz và nhiễu trắng Gaussian) để mô phỏng môi trường thực tế.
 - + Thiết kế bộ lọc: MATLAB được dùng để xác định các thông số kỹ thuật và thiết kế bộ lọc FIR bằng phương pháp cửa sổ Hamming. Nó tính toán các hệ số lý tưởng và thực hiện lượng tử hóa chúng sang định dạng số nguyên (Q1.15) để phù hợp với phần cứng.

- + Tiền xử lý và hậu xử lý dữ liệu: Tín hiệu sau khi tạo ra được lượng tử hóa sang định dạng 8-bit (Q1.7) và xuất sang file Hexadecimal để nạp vào môi trường mô phỏng Verilog. Sau khi Verilog xử lý xong, MATLAB sẽ đọc lại kết quả để khôi phục dạng sóng analog và tính toán các chỉ số sai số như RMSE và MAE nhằm đánh giá hiệu năng.
- Vai trò và chức năng của Verilog: Verilog được sử dụng để hiện thực hóa cấu trúc bộ lọc ở mức độ phần cứng thông qua ngôn ngữ mô tả phần cứng Verilog HDL. Các chức năng chính gồm:
 - + Xây dựng lõi xử lý FIR (Core Logic): Phần mềm thực thi các khối chức năng cốt lõi của bộ lọc bao gồm:
 - Thanh ghi dịch (Shift Register): Lưu trữ và dịch chuyển tuần tự các mẫu tín hiệu đầu vào qua từng chu kỳ xung nhịp.
 - Khối nhân tích lũy (MAC Unit): Thực hiện phép tính nhân chập giữa tín hiệu đầu vào và hệ số bộ lọc đã nạp.
 - Khối bão hòa (Scaling & Saturation): Đảm bảo tín hiệu đầu ra không bị tràn số và chuẩn hóa về định dạng Q1.15.
 - + Mô phỏng chức năng (Functional Verification): Thông qua Testbench, phần mềm đọc dữ liệu từ MATLAB, thực hiện tính toán lọc nhiễu chính xác từng bit (bit-true) và xuất kết quả ra file để kiểm chứng.
- Sự phối hợp giải quyết vấn đề: Sự kết hợp này tạo ra một quy trình khép kín: MATLAB thiết kế và cung cấp dữ liệu mẫu → Verilog mô phỏng cách thức vận hành của chip FPGA thực tế → MATLAB đánh giá lại độ chính xác. Quy trình này đảm bảo bộ lọc FIR thiết kế theo công nghệ FPGA không chỉ loại bỏ nhiễu quả nhiễu tần số cao mà còn giữ nguyên được hình dạng sóng ECG (đặc tính pha tuyến tính), giúp bảo toàn các thông tin quan trọng cho việc chẩn đoán.

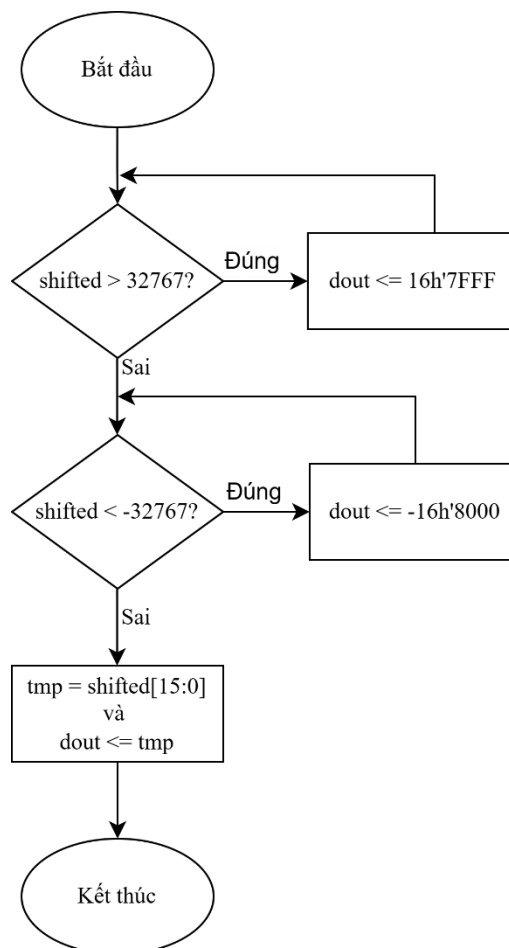
3.2.2 LƯU ĐỒ GIẢI THUẬT



Hình 3.3 Lưu đồ giải thuật tổng quan của hệ thống



Hình 3.4 Lưu đồ giải thuật khối nhân chập



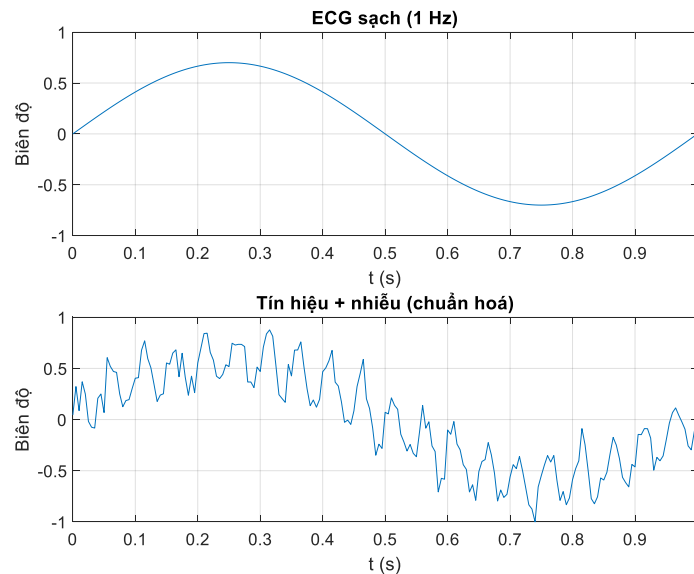
Hình 3.5 Lưu đồ giải thuật khối giới hạn biên

CHƯƠNG 4: KẾT QUẢ

4.1 HOẠT ĐỘNG CỦA HỆ THỐNG

4.1.1 MÔ HÌNH GIẢ LẬP TÍN HIỆU ECG

- Tín hiệu được xây dựng bao gồm ba thành phần: tín hiệu gốc (1 Hz), nhiễu cấu trúc (20 Hz) và nhiễu trắng (Gaussian noise)

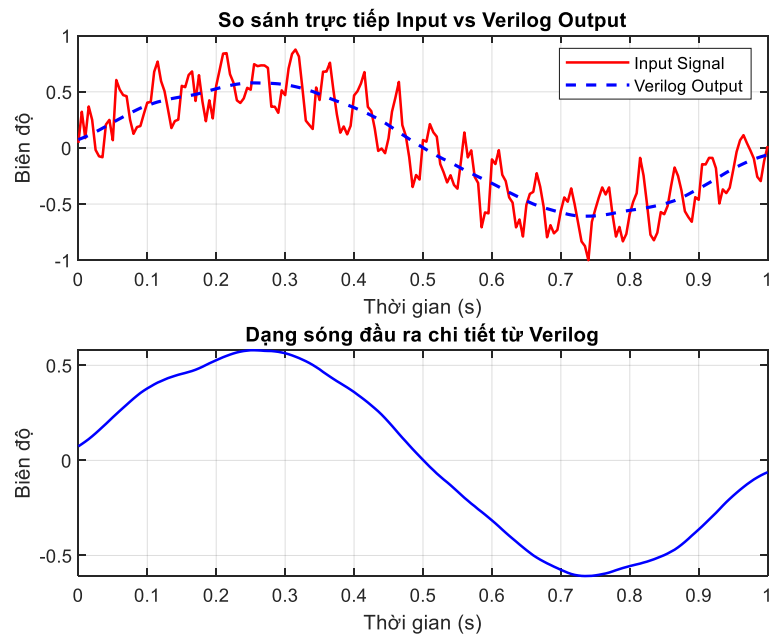
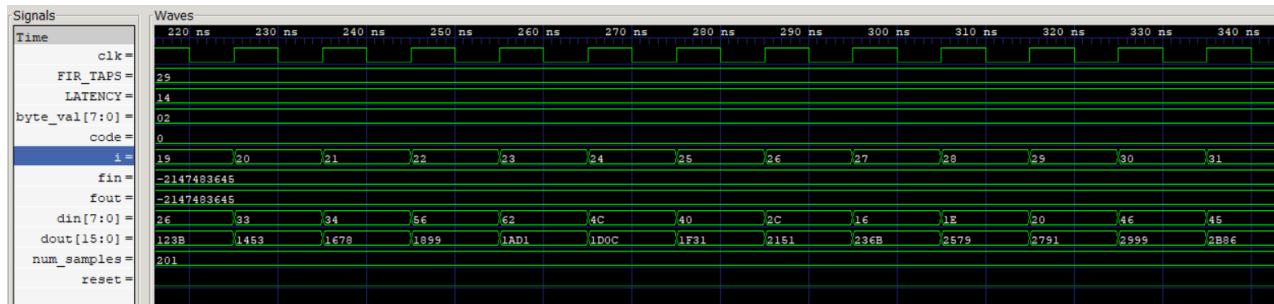


Hình 4.1 Tín hiệu giả lập ECG

4.1.2 MÔ PHỎNG TÍN HIỆU ĐẦU RA

4.1.2.1 MÔ PHỎNG TÍN HIỆU ĐẦU RA VỚI VERILOG

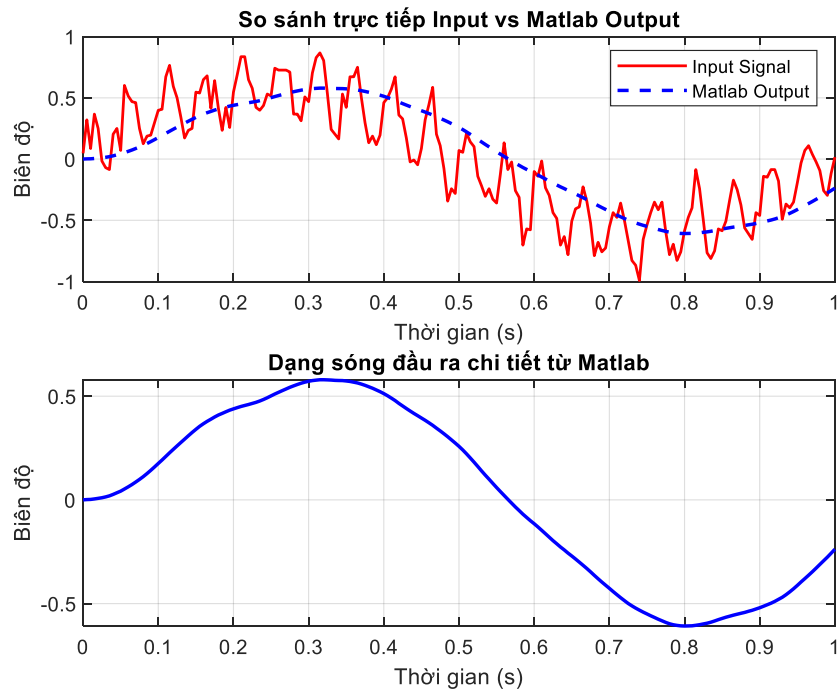
- Kiểm chứng chức năng (Functional Verification): Xác thực hoạt động của mô-đun FIR nhằm đảm bảo tính đúng đắn so với logic thiết kế. Quá trình này tập trung kiểm tra việc dịch chuyển chính xác các mẫu dữ liệu đầu vào qua hệ thống thanh ghi, cũng như tính chuẩn xác của các thao tác nhân - cộng tích lũy (MAC).
- Phân tích đặc tính thời gian: Xác định chính xác độ trễ (latency) của phần cứng, được tính từ thời điểm đưa mẫu dữ liệu đầu tiên vào hệ thống cho đến khi nhận được kết quả đầu ra hợp lệ.
- Kiểm soát tính toàn vẹn dữ liệu: Giám sát dạng sóng tín hiệu số nhằm đảm bảo không xảy ra hiện tượng tràn số (overflow) hoặc sai sót về bit dấu (sign bit) trong suốt quá trình xử lý với định dạng số nguyên có dấu 16-bit.



Hình 4.2 Tín hiệu giả lập sau khi thông qua bộ lọc (Verilog)

4.1.2.2 MÔ PHỎNG TÍN HIỆU ĐẦU RA VỚI MATLAB

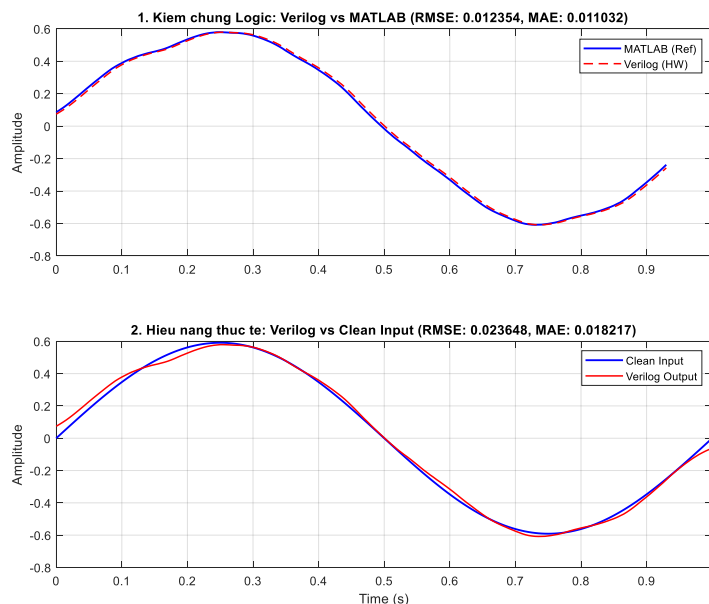
- Phần này hướng đến việc thiết kế bộ lọc FIR dưới dạng Fixed-point, đảm bảo độ chính xác bit-true so với mô hình Verilog nhằm phục vụ làm chuẩn so sánh.



Hình 4.3 Tín hiệu giả lập sau khi thông qua bộ lọc (Matlab)

4.1.3 SO SÁNH KẾT QUẢ

- Mục tiêu của phần này là xác minh độ chính xác của phần cứng (Verilog) so với mô hình lọc lý thuyết (MATLAB) và tín hiệu gốc.



Hình 4.4 So sánh đánh giá kết quả

4.2 ĐÁNH GIÁ PHÂN TÍCH HỆ THỐNG

4.2.1 PHÂN TÍCH KẾT QUẢ DỰA TRÊN SAI SỐ RMSE/MAE

Phương pháp đánh giá: thông qua hai chỉ số sai số phổ biến là sai số bình phương trung bình căn bậc 2 (RMSE) và sai số tuyệt đối trung bình (MAE) để so sánh, đánh giá kết quả giữa đầu ra bộ lọc Verilog (y_{ver}) và đầu ra bộ lọc tham chiếu MATLAB (y_{mat}) [9].

$$\text{Công thức RMSE: } RMSE = \sqrt{\frac{1}{L} \sum_{n=1}^L (y_{ver}(n) - y_{mat}(n))^2} \quad 4.0$$

$$\text{Công thức MAE: } MAE = \frac{1}{L} \sum_{n=1}^L |y_{ver}(n) - y_{mat}(n)| \quad 4.1$$

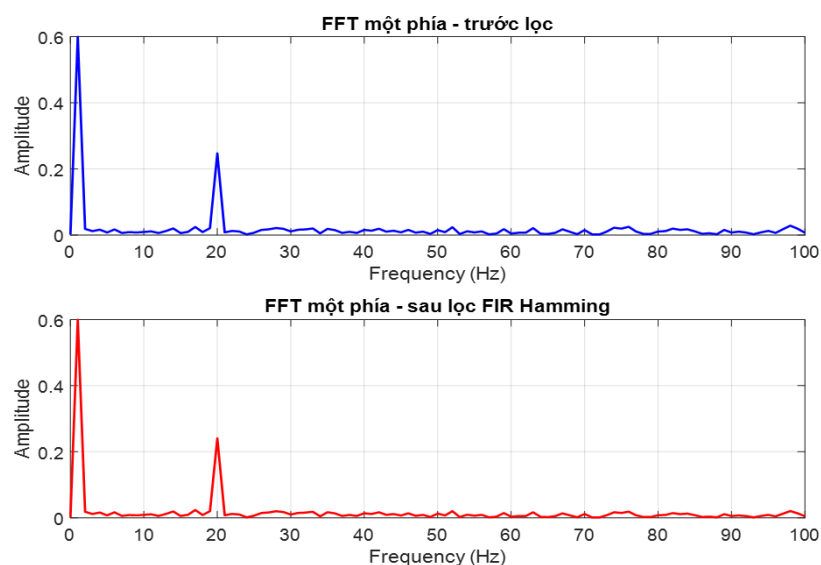
	Verilog và Matlab	Verilog và tín hiệu gốc
RMSE	0.012354	0.023648
MAE	0.011032	0.018217
Độ trễ pha	0	14

Bảng 4.1 Bảng so sánh sai số

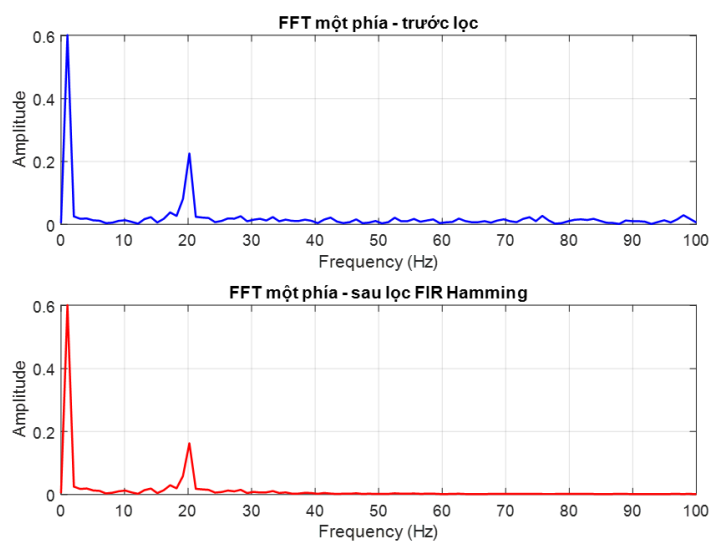
- Về độ chính xác:
 - + Với $RMSE \approx 1.2\%$ và $MAE \approx 1.1\%$ cho thấy bộ lọc đã hoạt động đúng theo lý thuyết.
 - + Với $RMSE \approx 2.4\%$ và $MAE \approx 1.8\%$ cho thấy bộ lọc đã thành công khôi phục phần lớn tín hiệu ECG gốc, qua đó chứng minh 29 taps đủ để vừa đảm bảo việc sử dụng tài nguyên hợp lý trên phần cứng vừa duy trì hiệu năng lọc.
- Về đặc tính thời gian: Độ trễ pha của hệ thống đạt chính xác 14 mẫu, hoàn toàn đồng nhất với cơ sở lý thuyết của bộ lọc FIR 29 taps: $\text{Delay} = (N-1)/2$. Khoảng trễ cố định 70ms này là một hằng số đối với mọi thành phần tần số, giúp bảo toàn nguyên vẹn hình dạng của sóng ECG giả lập.
- Hiệu quả khử nhiễu: Phân tích đồ thị kết quả cho thấy các thành phần nhiễu tần số cao đã bị triệt tiêu một cách rõ rệt. Tín hiệu ngõ ra có độ hội tụ rất cao so với tín hiệu sạch lý tưởng, cho thấy bộ lọc vận hành ổn định và đạt hiệu suất tốt trong môi trường chịu tác động của nhiễu hỗn hợp.

4.2.2 PHÂN TÍCH CẤU HÌNH BỘ LỌC

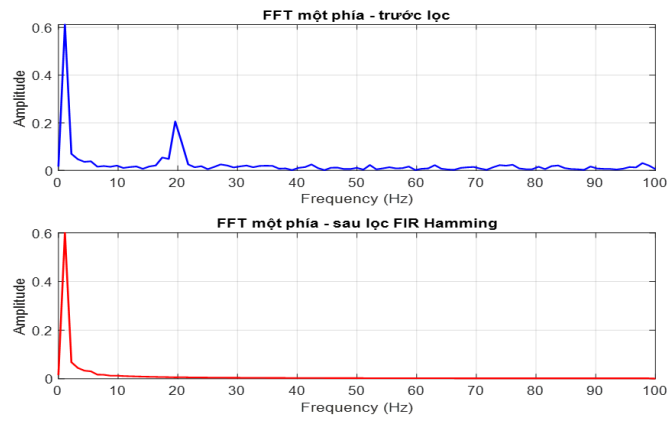
Nhằm tối ưu hóa cấu hình phần cứng cho bộ lọc FIR, em đã thực hiện khảo sát thay đổi số lượng Taps (hệ số). Hoạt động này giúp phân tích sự ảnh hưởng của hệ số về khả năng khử nhiễu, dung lượng tài nguyên sử dụng cùng độ trễ hệ thống.



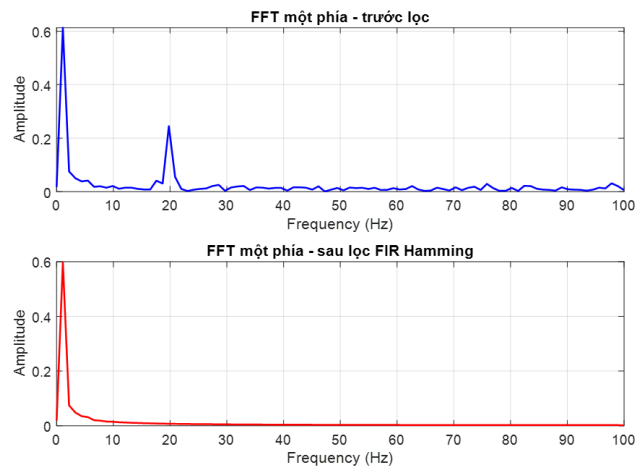
Hình 4.5 Biến đổi FFT trước và sau đối với bộ lọc 3 taps



Hình 4.6 Biến đổi FFT trước và sau đối với bộ lọc 5 taps



Hình 4.7 Biến đổi FFT trước và sau đối với bộ lọc 35 taps



Hình 4.8 Biến đổi FFT trước và sau đối với bộ lọc 39 taps

Số lượng Taps	RMSE (vs Clean)	MAE (vs Clean)	Độ trễ (ms)	Đánh giá khả năng lọc
3 Taps	0.200433	0.168301	5 ms	Nhiều vẫn còn rất lớn, dạng sóng bị biến dạng mạnh.
5 Taps	0.174419	0.147450	10 ms	Bắt đầu định hình sóng nhưng răng cưa nhiều còn dày.
7 Taps	0.084809	0.073305	15 ms	Khả năng lọc tốt, đáp ứng được yêu cầu cơ bản
29 Taps	0.080226	0.070004	70 ms	Điểm tối ưu: Sai số thấp nhất, tín hiệu tương đối mịn.
35 Taps	0.085573	0.077682	85 ms	Sai số bắt đầu tăng do ảnh hưởng của trễ pha.

39 Taps	0.087528	0.079524	95 ms	Trễ pha lớn làm lệch tín hiệu so với mốc tham chiếu.
---------	----------	----------	-------	--

Bảng 4.2 Bảng so sánh kết quả bộ lọc

Dựa trên các phân tích nêu trên, cấu hình 29 Taps được chọn là thông số thiết kế cuối cùng cho module Verilog vì hai lý do chính:

- Độ chính xác vượt trội: Cấu hình này cho thấy giá trị RMSE thấp nhất (0.080226) so với các cấu hình còn lại.
- Hiệu quả sử dụng tài nguyên: Việc nâng hệ số cao hơn (35–39 Taps) đòi hỏi thêm bộ nhân (Multiplier) và thanh ghi, trong khi không mang lại sự cải thiện quá lớn - do đó việc nâng hệ số lên cao hơn gây lãng phí tài nguyên không cần thiết.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Sau quá trình nghiên cứu lý thuyết, tính toán thiết kế và mô phỏng hệ thống, các mục tiêu đề ra của đề án "Thiết kế và mô phỏng bộ lọc FIR thông thấp ứng dụng xử lý tín hiệu ECG trên FPGA" đã được hoàn thành. Cụ thể, các kết quả chính đạt được như sau:

- Về mặt lý thuyết:
 - + Đã tiến hành nghiên cứu chuyên sâu về đặc tính của tín hiệu điện tâm đồ (ECG) và các nguồn nhiễu ảnh hưởng trực tiếp đến chất lượng tín hiệu, bao gồm nhiễu điện lưới và nhiễu cơ EMG.
 - + Nền tảng toán học của bộ lọc số FIR được nắm vững, đặc biệt là phương pháp thiết kế sử dụng hàm cửa sổ (Windowing) nhằm tối ưu hóa đặc tuyến tần số của bộ lọc.
 - + Quy trình chuyển đổi từ miền số thực trên MATLAB sang biểu diễn số nguyên (Fixed-point) được hiểu và vận dụng đúng đắn, đảm bảo tính tương thích với kiến trúc xử lý của FPGA.
- Về mặt thiết kế hệ thống:
 - + Bộ lọc FIR thông thấp (Low-pass Filter) đã được thiết kế thành công với tần số cắt xác định, trong đó cửa sổ Hamming được lựa chọn nhằm đạt độ suy giảm dải chặn trên -50 dB.
 - + Sơ đồ RTL được xây dựng bằng ngôn ngữ mô tả phần cứng Verilog, bao gồm đầy đủ các khối chức năng thiết yếu như thanh ghi dịch, bộ nhân và bộ cộng dồn.
 - + Kỹ thuật lượng tử hóa hệ số (8-bit/16-bit) được áp dụng nhằm tối ưu hóa việc sử dụng tài nguyên phần cứng, đồng thời vẫn duy trì sai số trong ngưỡng chấp nhận được.
- Về mặt kiểm chứng:
 - + Quá trình mô phỏng trên Verilog/Matlab cho thấy tín hiệu đầu ra đã loại bỏ thành công các thành phần nhiễu tần số cao, dạng sóng ECG thu được trở nên mượt mà và rõ nét.

- + Đáp ứng xung và đáp ứng pha của hệ thống đều thể hiện tính tuyến tính, đảm bảo các thành phần sóng đặc trưng (P, QRS, T) không bị méo dạng - đây là yêu cầu then chốt trong ứng dụng chẩn đoán lâm sàng.

Bên cạnh những kết quả đạt được, đồ án vẫn còn một số hạn chế nhất định cần được khắc phục trong thời gian tới:

- Tài nguyên phần cứng: Khi hệ số lọc tăng cao, số lượng bộ nhân (Multipliers/DSP Slices) cần thiết trên FPGA cũng tăng theo đáng kể. Hiện tại, các kỹ thuật tối ưu hóa diện tích như cấu trúc Distributed Arithmetic (DA) hay Canonic Signed Digit (CSD) chưa được khai thác và áp dụng một cách triệt để.
- Khả năng thích nghi: Do bộ lọc FIR được thiết kế với hệ số cố định (Fixed coefficients), hệ thống chỉ xử lý hiệu quả các loại nhiễu đã biết trước. Với những nhiễu biến đổi theo thời gian hoặc có tần số nằm trong dải thông của tín hiệu, hiệu suất lọc sẽ bị giảm sút đáng kể.
- Thực nghiệm lâm sàng: Phạm vi kiểm chứng của đồ án hiện mới chỉ dừng lại ở mức mô phỏng trên dữ liệu giả lập chưa được đánh giá trên tín hiệu ECG thu thập trực tiếp từ bệnh nhân trong môi trường nhiễu thực tế.

5.2 HƯỚNG PHÁT TRIỂN

Từ những kết quả và hạn chế đã được chỉ ra, dưới đây là một số đề xuất về hướng phát triển tiếp theo nhằm hoàn thiện hệ thống và nâng cao tính ứng dụng thực tiễn của đề tài:

- Tối ưu hóa kiến trúc phần cứng:
 - + Nghiên cứu và áp dụng kiến trúc bán song song (Semi-parallel) kết hợp với kỹ thuật chia sẻ tài nguyên (Resource Sharing) nhằm giảm thiểu số lượng bộ nhân, từ đó thu gọn diện tích logic và hạ thấp mức tiêu thụ năng lượng của hệ thống.
 - + Khảo sát các cấu trúc lọc tiên tiến theo hướng Multiplier-less FIR, trong đó phép nhân được thay thế bằng tổ hợp các phép dịch bit và cộng, góp phần tăng tốc độ xử lý và giảm độ phức tạp phần cứng.
- Phát triển bộ lọc thích nghi (Adaptive Filter):
 - + Mở rộng nghiên cứu sang lĩnh vực lọc thích nghi bằng cách triển khai các thuật toán như Least Mean Squares (LMS) hoặc Recursive Least Squares

(RLS). Các thuật toán này có khả năng tự động hiệu chỉnh hệ số bộ lọc theo sự biến động của nhiễu môi trường, mang lại hiệu quả vượt trội trong việc khử nhiễu trôi đường nền và các loại nhiễu cử động có đặc tính phi tuyến.

- Tích hợp hệ thống trên chip (System-on-Chip):
 - + Tích hợp bộ lọc FIR vào một nền tảng SoC hoàn chỉnh trên FPGA - chẳng hạn thông qua vi xử lý nhúng Zynq hoặc MicroBlaze - nhằm xây dựng một hệ thống xử lý tín hiệu ECG khép kín và có khả năng triển khai thực tế.
 - + Kết hợp với các thuật toán hậu xử lý (Post-processing) để thực hiện phát hiện phức bộ QRS và cảnh báo tự động các rối loạn nhịp tim ngay trên thiết bị, hướng tới ứng dụng trong hệ thống giám sát sức khỏe theo thời gian thực.

TÀI LIỆU THAM KHẢO

- [1] L. Sörnmo and P. Laguna, "Electrocardiogram (ECG) Signal Processing," John Wiley & Sons, 2006.
- [2] Y. Jia et al., "Preprocessing and Denoising Techniques for Electrocardiography and Magnetocardiography: A Review," *Bioengineering*, vol. 11, no. 11, p. 1109, Nov. 2024.
- [3] A. Gacek and W. Pedrycz, *ECG Signal Processing, Classification and Interpretation: A Comprehensive Framework of Computational Intelligence*, Springer-Verlag, 2012.
- [4] K. Aboutabikh and M. Yousef, "Design and implementation of a FIR digital low pass filter (LPF) using FPGA," *International Multidisciplinary Research Journal Reviews (IMRJR)*, vol. 2, no. 4, pp. 1–9, Apr. 2025.
- [5] M. A. A. Al-Dulaimi, H. A. Wahhab, and A. A. Amer, "Design and implementation of communication digital FIR filter for audio signals on the FPGA platform," *Journal of Communications*, vol. 18, no. 2, pp. 89–96, Feb. 2023.
- [6] TS. Trần Văn Hưng, *Thiết kế bộ lọc số FIR*, Trường Đại học Giao thông Vận tải, 2023.
- [7] I. A. Ozkan and A. Saday, "Design and implementation of FIR filter based on FPGA," *Proc. 7th Int. Conf. Advanced Technologies (ICAT'18)*, Apr.–May 2018.
- [8] B. Rashidi et al., "Low power FPGA implementation of digital FIR filter based on low power multiplexer base shift/add multiplier," *International Journal of Computer Theory and Engineering*, vol. 5, no. 2, pp. 346–350, Apr. 2013.
- [9] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Prentice Hall, 2007.
- [10] MathWorks, *Fixed-Point Filter Design in MATLAB*, MathWorks, 2025.
- [11] Xilinx Inc., *FIR Compiler v7.2 LogiCORE IP Product Guide (PG149)*, Xilinx Inc., Oct. 2022.

PHỤ LỤC

1. TẠO TÍN HIỆU GIẢ LẬP ECG

Code sẽ xuất ra file hex dưới định ‘ECG_Input.txt’ để làm tín hiệu đầu vào cho phụ lục 2 và 3.

```
% ECG giả lập + nhiễu, Q1.7 signed 8-bit, xuất input hex

clear; clc;

Fs = 200;                % Hz
t = 0:1/Fs:1;            % 1 giây
N = length(t);

% ECG gốc sạch (1 Hz)
rng(0);
ecg_clean_true = 0.7*sin(2*pi*1*t);

% Thêm nhiễu cấu trúc 20 Hz
ecg_with_noise = ecg_clean_true + 0.3*sin(2*pi*20*t);

% Thêm nhiễu Gaussian
noise = 0.1*randn(1,N);

% Tín hiệu input cuối cùng
signal = ecg_with_noise + noise;

% Chuẩn hoá biên độ
signal = signal / max(abs(signal));

% Q1.7
signal_q7 = int8(round(signal * 127));

% Xuất hex
fid = fopen('ECG_Input','w');
for i = 1:N
    fprintf(fid,'%02X\n', typecast(signal_q7(i),'uint8'));
end
fclose(fid);

disp('Generated ECG_Input.txt');
% =====
% VẼ TÍN HIỆU
% =====
figure;

subplot(2,1,1);
plot(t, ecg_clean_true);
title('ECG sạch (1 Hz)'); xlabel('t (s)'); ylabel('Biên độ'); grid on;

subplot(2,1,2);
plot(t, signal);
title('Tín hiệu + nhiễu (chuẩn hoá)'); xlabel('t (s)'); ylabel('Biên độ');
grid on;
```

2. BỘ LỌC FIR THAM CHIẾU TRÊN MATLAB

Kết quả của bộ lọc được lưu dưới tên 'Output_Reference.txt' để tiến hành so sánh

```
clear; clc;

%% --- 1. Cấu hình & Đọc dữ liệu ngõ vào ---
Fs = 200; fc = 3; N_TAPS = 29;

fid = fopen('ECG_Input.txt', 'r');
assert(fid > 0, 'Không mở được file input');
C = textscan(fid, '%s'); fclose(fid);

signal_q7 = typecast(uint8(hex2dec(C{1})), 'int8'); % Đọc Q1.7
signal_float = double(signal_q7) / 128;
N = length(signal_q7);
t = (0:N-1)/Fs;

%% --- 2. Thiết kế bộ lọc & Lọc Bit-True ---
h_float = fir1(N_TAPS-1, fc/(Fs/2), 'low', hamming(N_TAPS));
h_float = h_float / sum(h_float); % DC gain = 1
h_q15 = int16(round(h_float * 32767));

% Lọc convolution dùng filter() thay cho vòng lặp for phức tạp (vẫn giữ đúng logic bit-true)
x_padded = [zeros(N_TAPS-1, 1); double(signal_q7)];
acc = filter(double(h_q15), 1, x_padded);
acc = acc(N_TAPS:end); % Bỏ phần đệm đầu

% Dịch bit (arithmetic shift right 7) và bão hòa (Saturate) về int16
ytmp = floor(acc / 2^7);
ytmp(ytmp > 32767) = 32767;
ytmp(ytmp < -32767) = -32767;
y_ref = int16(ytmp);
y_float = double(y_ref) / 32767;

%% --- 3. Ghi dữ liệu ngõ ra (Hex) ---
fid = fopen('Output_Reference.txt', 'w');
assert(fid > 0, 'Không mở được file output');
fprintf(fid, '%04X\n', mod(double(y_ref), 65536));
fclose(fid);
fprintf('Đã xử lý xong %d mẫu.\n', N);

%% --- 4. Hiển thị Trực quan ---
% Figure 1: So sánh tín hiệu miền thời gian và tần số (FFT)
figure('Name', 'Phân tích Tín hiệu', 'Color', 'w');
tiledlayout(2, 2, 'TileSpacing', 'compact');

nexttile; plot(t, signal_float); grid on;
title('ECG Ngõ vào (Gốc + Nhiễu)'); ylabel('Amp');

nexttile; plot(t, y_float, 'r'); grid on;
title('ECG Ngõ ra (Sau lọc FIR)'); ylabel('Amp');

% Tính FFT nhanh để vẽ luôn
f = Fs * (0:floor(N/2)) / N;
Xf = abs(fft(signal_float)/N); Yf = abs(fft(y_float)/N);
```

```

nexttile; plot(f, 2*Xf(1:floor(N/2)+1)); grid on;
title('FFT Trước lọc'); xlabel('Freq (Hz)');

nexttile; plot(f, 2*Yf(1:floor(N/2)+1), 'r'); grid on;
title('FFT Sau lọc'); xlabel('Freq (Hz)');

% Figure 2: Đặc tính bộ lọc FIR
figure('Name', 'Đặc tính Bộ lọc', 'Color', 'w');
tiledlayout(3, 1, 'TileSpacing', 'compact');

nexttile; stem(0:N_TAPS-1, h_float, 'filled'); grid on;
title('Đáp ứng xung h[n]');

[H, F] = freqz(h_float, 1, 1024, Fs);
nexttile; plot(F, 20*log10(max(abs(H), 1e-12))); grid on;
title('Đáp ứng biên độ (dB)');

nexttile; plot(F, unwrap(angle(H))); grid on;
title('Đáp ứng pha (rad)'); xlabel('Frequency (Hz)');

%% --- 5. Lưu Workspace ---
save('filter_analysis.mat', 'Fs', 'fc', 'N_TAPS', 'h_q15', 'signal_q7',
'y_ref');

```

3. BỘ LỌC FIR VERILOG

Kết quả của bộ lọc được lưu dưới tên ‘Output_V.txt’ để tiến hành so sánh

Code module:

```

`timescale 1ns/1ns
module fir_filter (
    input wire      clk,
    input wire      reset,
    input wire signed [7:0] din,    // Q1.7
    output reg signed [15:0] dout   // Q1.15
);

    parameter N_TAPS = 29;

    // =====
    // FIR coefficients Q1.15 (fc = 3 Hz, COPY từ MATLAB)
    // =====
    reg signed [15:0] h [0:N_TAPS-1];
    initial begin
        h[0] = 16'h0085; h[1] = 16'h009F; h[2] = 16'h00E3; h[3] =
16'h0152;
        h[4] = 16'h01EB; h[5] = 16'h02A8; h[6] = 16'h0382; h[7] =
16'h046E;
        h[8] = 16'h0560; h[9] = 16'h064A; h[10] = 16'h0721; h[11] =
16'h07D6;

```

```

        h[12] = 16'h0861; h[13] = 16'h08B7; h[14] = 16'h08D4; h[15] =
16'h08B7;
        h[16] = 16'h0861; h[17] = 16'h07D6; h[18] = 16'h0721; h[19] =
16'h064A;
        h[20] = 16'h0560; h[21] = 16'h046E; h[22] = 16'h0382; h[23] =
16'h02A8;
        h[24] = 16'h01EB; h[25] = 16'h0152; h[26] = 16'h00E3; h[27] =
16'h009F;
        h[28] = 16'h0085;
    end

    // =====
    // Shift register (Q1.7)
    // =====
    reg signed [7:0] sr [0:N_TAPS-1];
    integer i;

    // =====
    // Multiply-Accumulate + Saturation
    // =====
    reg signed [31:0] acc;
    reg signed [31:0] shifted;
    reg signed [15:0] tmp;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            for (i = 0; i < N_TAPS; i = i + 1)
                sr[i] <= 0;
            dout <= 0;
        end else begin
            // shift register
            for (i = N_TAPS-1; i > 0; i = i - 1)
                sr[i] <= sr[i-1];
            sr[0] <= din;

            // multiply-accumulate
            acc = 0;
            for (i = 0; i < N_TAPS; i = i + 1)
                acc = acc + sr[i] * h[i]; // giữ đúng thứ tự hệ số

            // scale Q1.7 * Q1.15 = Q2.22 → Q1.15
            shifted = acc >>> 7;

            // saturation
            if (shifted > 32767)
                dout <= 16'sh7FFF;

```

```

        else if (shifted < -32767)
            dout <= -16'sh8000;
        else begin
            tmp = shifted[15:0];
            dout <= tmp;
        end
    end
end
end

endmodule

```

Testbench:

```

`timescale 1ns/1ns
module fir_tb;

    reg clk = 0;
    always #5 clk = ~clk;    // 10 ns period

    reg reset;
    reg signed [7:0] din;
    wire signed [15:0] dout;

    fir_filter dut (
        .clk(clk),
        .reset(reset),
        .din(din),
        .dout(dout)
    );

    integer fin, fout;
    reg [7:0] byte_val;
    reg signed [7:0] mem[0:200000];
    integer i, num_samples, code;

    localparam FIR_TAPS = 29;
    localparam LATENCY = (FIR_TAPS-1)/2; // = 14

    initial begin
        // dump waveform
        $dumpfile("fir_wave.vcd");
        $dumpvars(0, fir_tb);

        // --- đọc input hex ---
        $display("FIR TB: reading input file...");
        fin = $fopen("ECG_Input.txt", "r");
    end
endmodule

```

```

if (fin == 0) begin
    $display("Cannot open input file");
    $finish;
end

i = 0;
code = $fscanf(fin, "%h", byte_val);
while (code == 1) begin
    mem[i] = $signed(byte_val); // Q1.7 signed
    i = i + 1;
    code = $fscanf(fin, "%h", byte_val);
end
fclose(fin);
num_samples = i;
$display("Loaded %0d samples", num_samples);

fout = $fopen("Output_V.txt", "w");

// --- reset & prime ---
reset = 1;
din = 0;
repeat (3) @(posedge clk);
reset = 0;

// --- main processing: drive trước, ghi sau LATENCY ---
for (i = 0; i < num_samples; i = i + 1) begin
    // lái input cho chu kỳ kế tiếp
    din = mem[i];
    @(posedge clk);

    // sau khi qua đủ LATENCY chu kỳ, dout mới hợp lệ để ghi
    if (i >= LATENCY)
        $fwrite(fout, "%04X\n", dout[15:0]); // signed 16-bit hex
end

// --- flush pipeline: thêm LATENCY chu kỳ để xả hết ---
for (i = 0; i < LATENCY; i = i + 1) begin
    din = 0; // hoặc giữ mem[num_samples-1], tùy bạn
    @(posedge clk);
    $fwrite(fout, "%04X\n", dout[15:0]);
end

fclose(fout);
$display("Simulation output written to Output_V.txt");
$finish;
end

```

```
endmodule
```

4. SO SÁNH HIỆU NĂNG

Tiến hành so sánh giữa ‘Output_V.txt’ tượng trưng cho bộ lọc chạy trên phần cứng với ‘Output_Reference.txt’ tượng trưng cho bộ lọc lý thuyết, và tín hiệu gốc chưa thêm nhiễu.

```
clear; clc; close all;

%% ===== 1. CẤU HÌNH & LOAD DATA =====
Fs = 200; N_TAPS = 29; delay = (N_TAPS-1)/2;

read_q15 = @(filename) double(typecast(uint16(hex2dec(textread(filename, '%s'))),
'int16')) / 32768;

y_matlab = read_q15('Output_Reference.txt');
y_verilog = read_q15('Output_V.txt');

N_total = length(y_verilog);
t_real = (0:N_total-1)/Fs;

%% ===== 2. CHUẨN HÓA BIÊN ĐỘ ĐẦU VÀO =====
ecg_clean_raw = 0.7 * sin(2*pi*1*t_real);
ecg_with_noise = ecg_clean_raw + 0.3 * sin(2*pi*20*t_real);
rng(0);
noise = 0.1 * randn(1, N_total);
signal_raw = ecg_with_noise + noise;

scale_factor = max(abs(signal_raw));
ecg_clean = ecg_clean_raw / scale_factor;

%% ===== 3. TÍNH TOÁN SAI SỐ KHỚP DỮ LIỆU =====
% Khớp pha Verilog vs MATLAB
y_mat_part = y_matlab(delay+1:end);
L_comp = min(length(y_mat_part), length(y_verilog));
y_mat_final = y_mat_part(1:L_comp);
y_ver_final = y_verilog(1:L_comp);

err_hw = y_ver_final - y_mat_final;
rmse_hw = sqrt(mean(err_hw.^2));
mae_hw = mean(abs(err_hw));

% Khớp pha Verilog vs Clean
L_real = min(length(y_verilog), length(ecg_clean));
y_v_real = y_verilog(1:L_real);
y_c_real = ecg_clean(1:L_real);

err_real = y_v_real - y_c_real;
rmse_real = sqrt(mean(err_real.^2));
mae_real = mean(abs(err_real));

%% ===== 4. VẼ ĐỒ THỊ ĐỐI CHIẾU (1 FIGURE) =====
figure('Name','Ket qua Kiem chung He thong FIR FPGA','Color','w','Position',[100,
100, 800, 650]);
 tiledlayout(2,1,'TileSpacing','loose','Padding','compact');

% Hàng 1: Verilog vs MATLAB (Kiểm chứng thiết kế phần cứng)
nexttile;
plot((0:L_comp-1)/Fs, y_mat_final, 'b', 'LineWidth', 1.5); hold on;
plot((0:L_comp-1)/Fs, y_ver_final, 'r--', 'LineWidth', 1.2); grid on;
```

```

title(sprintf('1. Kiem chung Logic: Verilog vs MATLAB (RMSE: %.6f, MAE: %.6f)',
rmse_hw, mae_hw));
legend('MATLAB (Ref)', 'Verilog (HW)'); ylabel('Amplitude');

% Hàng 2: Verilog vs Clean Input (Hiệu năng khôi phục tín hiệu)
nexttile;
plot((0:L_real-1)/Fs, y_c_real, 'b', 'LineWidth', 1.5); hold on;
plot((0:L_real-1)/Fs, y_v_real, 'r', 'LineWidth', 1.2); grid on;
title(sprintf('2. Hieu nang thuc te: Verilog vs Clean Input (RMSE: %.6f, MAE: %.6f)',
rmse_real, mae_real));
legend('Clean Input', 'Verilog Output'); xlabel('Time (s)'); ylabel('Amplitude');

%% ===== 5. IN KẾT QUẢ RA COMMAND WINDOW =====
fprintf('\n===== \n');
fprintf('KẾT QUẢ PHÂN TÍCH HỆ THỐNG ĐIỆN TİM GIÁ LẬP: \n');
fprintf('1. So voi Ly thuyet MATLAB (Kiem chung logic phan cung): \n');
fprintf('    - RMSE = %.6f \n    - MAE = %.6f \n', rmse_hw, mae_hw);
fprintf('2. Verilog so voi Clean Input (Kiem chung hieu nang thuc te): \n');
fprintf('    - RMSE = %.6f \n    - MAE = %.6f \n', rmse_real, mae_real);
fprintf('    - Do tre pha vat ly: %d mau (~%.1f ms) \n', delay, (delay/Fs)*1000);
fprintf('===== \n');

```